

RUST



Dušan Kolář

kolar@fit.vut.cz

<https://doc.rust-lang.org/book/title-page.html>

Content



- Hello world!
- Cargo world!
- Variables, scope/shadowing, atomic types
- Compound types
- Functions – simple ones
- Control flow
- Ownership
- Slices
- Data structures
- Enums
- Packages, ...
- Built-in collections
- Extras/errors, ...
- Generic types, ...
- Lifetimes
- Example – small app.

Content (continued)



- Functional features
- Smart Pointers
- Concurrency
- Asynchronous Operations
- OOP in Rust
- Pattern Matching
- The End

Hello World!



- Install
 - <https://doc.rust-lang.org/book/ch01-01-installation.html>
- Make suitable directory
 - See your OS doc. 😊
- Change your position there
- Create file **main.rs** in that directory

Hello World!



- **fn main() {
 println!("Hello, world!");
}**
- `rustc main.rs`
- `./main`
Hello, world!
- Done 😊

Cargo World!



- cargo – build system & package manager
- rust – **snake case** required!!
- cargo new hello_carg
- `Creating` binary (application) `hello\_cargo` package
`note`: see more `Cargo.toml` keys and their definitions at <https://doc.rust-lang.org/cargo/reference/manifest.html>
- Directory created, some files, directory src (look inside, edit main.rs)
- cargo build
- `Compiling` hello_cargo v0.1.0 (/home/kolar/Vyuka/FLP/Rust/hello_cargo)
`Finished` `dev` profile [unoptimized + debuginfo] target(s) in 0.44s
- target/debug/hello_cargo
Hello, cargo world!

Cargo World!



- cargo run
 - `Finished`dev` profile [unoptimized + debuginfo] target(s) in 0.02s`
`Running`target/debug/hello_cargo``
- Hello, cargo world!
- Edit main.rs
- cargo check
 - `Checking`hello_cargo` v0.1.0 (/home/kolar/Vyuka/FLP/Rust/hello_cargo)`
`Finished`dev` profile [unoptimized + debuginfo] target(s) in 0.11s`
- cargo build -release
 - `Compiling`hello_cargo` v0.1.0 (/home/kolar/Vyuka/FLP/Rust/hello_cargo)`
`Finished`release` profile [optimized] target(s) in 0.22s`
- `./target/release/hello_cargo`

Variables



- Variables?? 😊
 - The same as in ML languages (let in Haskell)
 - SINGLE Assignment of a value, e.g.
let apples = 5; // immutable
- Mutable values 🤔
 - The same as in ML languages (Haskell MVar)
 - Can be overwritten, e.g.
let mut bananas = 5; // mutable
- **println!("Number of bananas: {}", bananas);**
println!("Apples {apples}/{}}!", apples-2);

Mutables Usage



- ```
fn main() {
 let mut x = 5;
 println!("The value of x is: {x}");
 x = 6;
 println!("The value of x is: {x}");
}
```

-

# Constants



- Simple:  
**const THREE\_HOURS\_IN\_SECONDS: u32 = 60\*60\*3;**
- Note:
  - Type
  - Constant expression
- Of course, no mutable constant 😊

# Shadowing / Scope



- Hiding, nested blocks + shadowing
- ```
fn main() {  
    let x = 5;  
    let x = x + 1;  
    {  
        let x = x * 2;  
        println!("The value of x  
                in the inner scope is: {x}");  
    }  
    println!("The value of x is: {x}");  
}
```

Shadowing – Run



- **Compiling** Compiling `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.88s
Running `target/debug/shadowing`

The value of x in the inner scope is: 12

The value of x is: 6

Shadowing – Counterexample



- Fine, new variable
let spaces = " ";
let spaces = spaces.len();
- Wrong, type change
let mut spaces = " ";
spaces = spaces.len();
- Rust is rather like Haskell then Python and similar stuff

Basic Types



Length	Signed	Unsigned
8-bit	i8	u8
16-bit	i16	u16
32-bit	i32	u32
64-bit	i64	u64
128-bit	i128	u128
arch	isize	usize

Literals



Number literals	Example
Decimal	98_222
Hex	0xff
Octal	0o77
Binary	0b1111_0000
Byte (u8 only)	b'A'

Other Types



- Floating point: f64 (default), f32, C-like
- Boolean values: bool, true, false
- Character: char, 'a', '🐱'

Tuples



- Pretty Haskellish 😊
- ```
fn main() {
 let tup: (i32, f64, u8) = (500, 6.4, 1);
}
```

—
- ```
fn main() {  
    let tup = (500, 6.4, 1);  
    let (x, y, z) = tup;  
    println!("The value of y is: {y}");  
}
```

Tuples – continued



- ```
fn main() {
 let x: (i32, f64, u8) = (500, 6.4, 1);
 let five_hundred = x.0;
 let six_point_four = x.1;
 let one = x.2;
}
```

# Arrays



- ```
fn main() {  
    let a = [1, 2, 3, 4, 5];  
    let months = ["January", "February", "March", "April",  
                  "May", "June", "July", "August", "September",  
                  "October", "November", "December"];  
    let a: [i32; 5] = [1, 2, 3, 4, 5]; // type and size  
    let a = [3; 5]; // [3, 3, 3, 3, 3] - value and size  
  
    let first = a[0];  
    let second = a[1];  
}
```
- Simply always initialized...

Functions



- ```
fn main() {
 println!("Hello, world!");
 another_function();
}

fn another_function() {
 println!("Another function.");
}
```
- Just keep snake case...

# Function Parameters



- You MUST define type (rust creators decision)

- ```
fn main() {  
    print_labeled_measurement(5, 'h');  
}
```

```
fn print_labeled_measurement(value: i32, unit_label: char) {  
    println!("The measurement is: {value}{unit_label}");  
}
```

Expression vs. Statement



- Expression “returns” a value, statement not (state change)

- **fn main() {**

```
    let y = {  
        let x = 3;  
        x + 1  
    };
```

```
    println!("The value of y is: {y}");
```

```
}
```

Function Returning a Value



- ```
fn main() {
 let x = plus_one(5);
 println!("The value of x is: {x}");
}
```

```
fn plus_one(x: i32) -> i32 {
 x + 1 // no semicolon !!!
}
```

# Control Flow – if



- Almost C-like, no brackets needed

- ```
fn main() {  
    let number = 6;  
  
    if number % 4 == 0 {  
        println!("number is divisible by 4");  
    } else if number % 3 == 0 {  
        println!("number is divisible by 3");  
    } else if number % 2 == 0 {  
        println!("number is divisible by 2");  
    } else {  
        println!("number is not divisible by 4, 3, or 2");  
    }  
}
```


Control Flow – if



- Haskell:
 - **factorial** n | n < 0 = error “Negative input for factorial”
| n < 2 = 1
| True = n * factorial (n - 1)
- Rust:
 - **fn factorial(n:u64) -> u64 {
 if n<2 { 1 }
 else { n * factorial(n-1) }
}**

Example – Large Integers



- Yes! But rather later....

- Just a digest:

- **use num_bigint::BigUint;**
use num_bigint::ToBigUint;

```
fn main() {  
    println!("Factorial BIG of 25 = {}",fact(25));  
}
```

```
fn fact(n:u64) -> BigUint {  
    let one = (1 as usize).to_biguint().unwrap();  
    if n<2 { one }  
    else {  
        let mut init = (2 as usize).to_biguint().unwrap();  
        let mut accum = one.clone();  
        for _ in 1 .. n {  
            accum = accum * &init;  
            init = init + &one;  
        }  
        accum  
    }  
}
```

Large Integers – continued



- Seems horrible? Let's make it worse...

- ```
use num_bigint::BigUint;
use num_bigint::ToBigUint;
use std::cmp::Ordering;
```

```
fn main() {
 println!("Factorial BIG recursive of 5 = {}",
 factorial((5 as usize).to_biguint().unwrap()));
 println!("Factorial BIG recursive of 25 = {}",
 factorial((25 as usize).to_biguint().unwrap()));
}
```

```
fn factorial(n : BigUint) -> BigUint {
 if n.cmp(&(2 as usize).to_biguint().unwrap()) <= Ordering::Less {
 (1 as usize).to_biguint().unwrap()
 } else {
 let nc = n.clone();
 factorial(n - &(1 as usize).to_biguint().unwrap()) * &nc
 }
}
```

# Large Integers – forward recursion



- ```
fn factorial(n : BigUint) -> BigUint {  
    fact_fwd( (1 as usize).to_biguint().unwrap(), n )  
}
```



```
fn fact_fwd(accum: BigUint, n: BigUint) -> BigUint {  
    if n.cmp( &(2 as usize).to_biguint().unwrap() ) == Ordering::Less {  
        accum.clone()  
    } else {  
        fact_fwd( accum * &n,  n - &(1 as usize).to_biguint().unwrap() )  
    }  
}
```

Large Integers – last



- Recursion is not good, rust compiler doesn't change it to loops
- Thus:
- ```
fn factorial(mut n : BigUint) -> BigUint {
 let mut res = (1 as usize).to_biguint().unwrap();
 let one = (1 as usize).to_biguint().unwrap();
 while n.cmp(&(2 as usize).to_biguint().unwrap()) >= Ordering::Equal {
 res = res * &n;
 n = n - &one;
 }
 res
}
```
- But version with u64 is faster, if it is sufficient...
- Why?

# Loops



- Loop 😊
- ```
fn main() {  
    loop {  
        println!("again!");  
    }  
}
```
- Infinite loop...

Loops – Loop Labeling



```
• fn main() {  
    let mut count = 0;  
    'counting_up: loop {  
        println!("count = {count}");  
        let mut remaining = 10;  
  
        loop {  
            println!("remaining = {remaining}");  
            if remaining == 9 {  
                break;  
            }  
            if count == 2 {  
                break 'counting_up;  
            }  
            remaining -= 1;  
        }  
  
        count += 1;  
    }  
    println!("End count = {count}");  
}
```

Loop – while



- C style except brackets
- ```
fn main() {
 let a = [10, 20, 30, 40, 50];
 let mut index = 0;

 while index < 5 {
 println!("the value is: {}", a[index]);

 index += 1;
 }
}
```



# Loop – Collection Traversal



- FOR
- ```
fn main() {  
    let a = [10, 20, 30, 40, 50];  
  
    for element in a {  
        println!("the value is: {element}");  
    }  
}
```

Loop – FOR int



- ```
fn main() {
 for number in 2..5 {
 print!("{number}\t");
 }
 println!("");
 for number in (1..4).rev() {
 println!("{number}!");
 }
 println!("LIFTOFF!!!");
}
```

# Output...



- **Compiling** for\_num v0.1.0 (/home/kolar/Vyuka/FLP/Rust/for\_num)  
**Finished** `dev` profile [unoptimized + debuginfo] target(s) in 0.23s  
**Running** `target/debug/for\_num`

- 2            3            4

3!

2!

1!

LIFTOFF!!!

- Other **for** usage will come later – iterators over collections

# Ownership



- Citation:
- *Ownership is Rust's most unique feature and has deep implications for the rest of the language. **It enables Rust to make memory safety guarantees without needing a garbage collector**, so it's important to understand how ownership works. ...*
- A set of rules that must be obeyed, if not, program won't compile.
- The more you'll understand the more you'll get used to.

# Ownership – Review



- Stack – LIFO structure
  - Operations: push, pop – **simple, fast**
- Heap – "random" access
  - Operations: allocation – need a space of a certain size, must **find it**
- Data locality – data close together can be processed faster than jumping from one address to completely different
- Program stack – uses program to store data, pointers to heap, necessary information to make the program run

# Ownership – Rules



- Each value in Rust has an owner.
- There can only be one owner at a time.
- When the owner goes out of scope, the value will be dropped.

# Ownership – Variable Scope



- ```
{           // s is not valid here, it's not yet declared
  let s = "hello"; // s is valid from this point forward

                // do stuff with s
}
```

// this scope is now over, and s is no longer valid
- When **s** comes into scope, it is valid.
- It remains valid until it goes out of scope.
- *Note, s above references literal, cannot be changed:*
- ```
let mut s = String::from("hello");
s.push_str(", world!"); // push_str() appends a literal to a String
println!("{s}"); // This will print `hello, world!`
```

# Ownership – Memory Release



- ```
{  
    let s = String::from("hello");  
        // s is valid from this point forward  
  
        // do stuff with s  
  
}    // this scope is now over, and s is no  
    // longer valid
```
- Thus, the memory is released!
Rust – memory is **dropped** on **'}'**

Ownership – Values

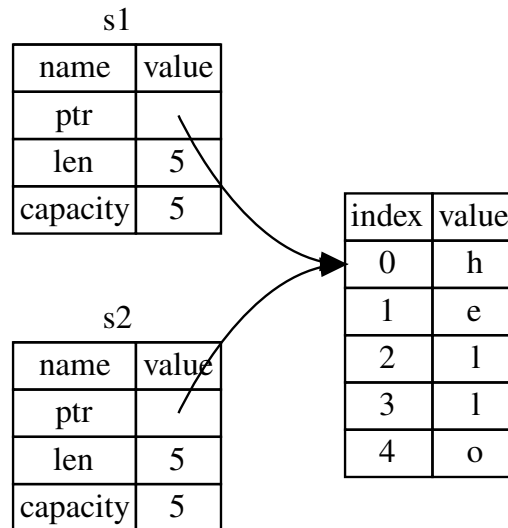
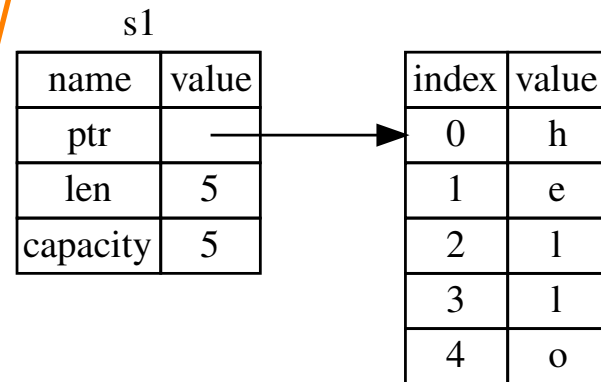


- `let x = 5;`
`let y = x; // copy of 5 - simple value`

- Easy, so far... 😊

- `let s1 = String::from("hello");`
`let s2 = s1;`

- Also copy, but only of “value”,
the structure
- BUT!! The process continues,
s1 value is no longer valid,
it's dropped



Ownership – Values

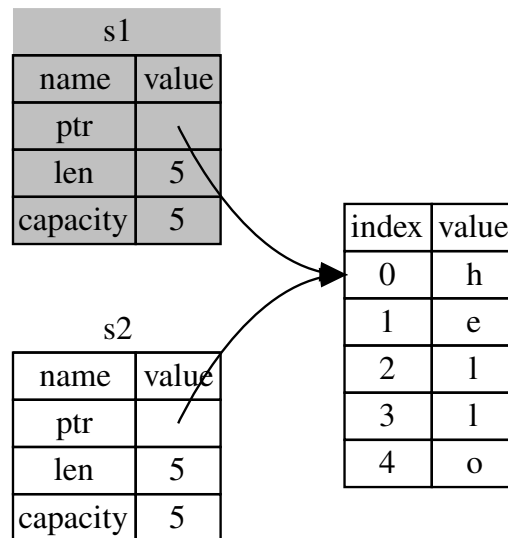
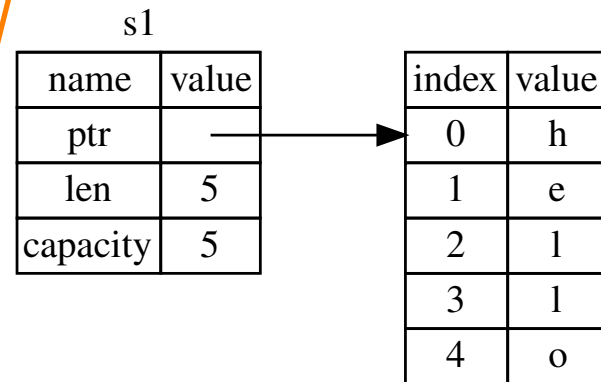


- `let x = 5;`
`let y = x; // copy of 5 - simple value`

- Easy, so far... 😊

- `let s1 = String::from("hello");`
`let s2 = s1;`

- Also copy, but only of “value”, the structure
- BUT!! The process continues, **s1 value is no longer valid**, it's dropped



Ownership – Scope & Assignment



- **let mut s = String::from("hello");
s = String::from("ahoy");
println!("{s}, world!");**
- Prints “ahoy, world!”
- Referred data on heap are dropped immediately

s

name	value
ptr	→
len	4
capacity	4

index	value
0	a
1	h
2	o
3	y

index	value
0	h
1	e
2	l
3	l
4	o

Ownership – Cloning



- ```
let s1 = String::from("hello");
let s2 = s1.clone();
println!("s1 = {s1}, s2 = {s2}");
```
- Created a **deep** copy
- Calls **method** clone
- *On methods later*
- Strongly dependent on the implementation of cloning method

# Ownership – Copy Trait



- Trait – rys, vlastnost
  - (Haskell type class, or something like that, later)
- Simple, atomic types implement, types with similar features can have – those not referencing data on a heap
- E.g. integers, boolean values, floating point numbers, characters, and tuples over these
- That's why this works:  
**let x = 5;**  
**let y = x;**

# Ownership – Functions



- New scope introduction
- Same behavior based on *Copy* trait

```
fn main() {
 let s = String::from("hello"); // s comes into scope

 takes_ownership(s); // s's value moves into the function...
 // ... and so is no longer valid here

 let x = 5; // x comes into scope

 makes_copy(x); // because i32 implements the Copy trait,
 // x does NOT move into the function,
 println!("{}", x); // so it's okay to use x afterward

} // Here, x goes out of scope, then s. But because s's value was moved, nothing
 // special happens.

fn takes_ownership(some_string: String) { // some_string comes into scope
 println!("{}", some_string);
} // Here, some_string goes out of scope and `drop` is called. The backing
 // memory is freed.

fn makes_copy(some_integer: i32) { // some_integer comes into scope
 println!("{}", some_integer);
} // Here, some_integer goes out of scope. Nothing special happens.
```

# Ownership – Returning Values



- ```
fn main() {  
    let s1 = gives_ownership();  
    // gives_ownership moves its return  
    // value into s1  
  
    let s2 = String::from("hello");  
    // s2 comes into scope  
  
    let s3 = takes_and_gives_back(s2);  
    // s2 is moved into  
    // takes_and_gives_back, which also  
    // moves its return value into s3  
  
} // Here, s3 goes out of scope  
  // and is dropped. s2 was moved,  
  // so nothing happens. s1 goes out  
  // of scope and is dropped.
```

- ```
fn gives_ownership() -> String {
 // gives_ownership will move its
 // return value into the function
 // that calls it

 let some_string = String::from("yours");
 // some_string comes into scope

 some_string // some_string is returned and
 // moves out to the calling
 // function
}

// This function takes a String and returns one
fn takes_and_gives_back(a_string: String)
 -> String
{ // a_string comes into scope

 a_string // a_string is returned and
 // moves out to the calling
 // function
}
```

# Ownership – Value Transport



- ```
fn main() {  
    let s1 = String::from("hello");  
  
    let (s2, len) = calculate_length(s1);  
  
    println!("The length of '{s2}' is {len}.");  
}
```



```
fn calculate_length(s: String) -> (String, usize) {  
    let length = s.len();          // len() returns the length of a String  
  
    (s, length)  
}
```
- This is the most straightforward way
- As rust people say, too much ceremony 😊

Ownership – References

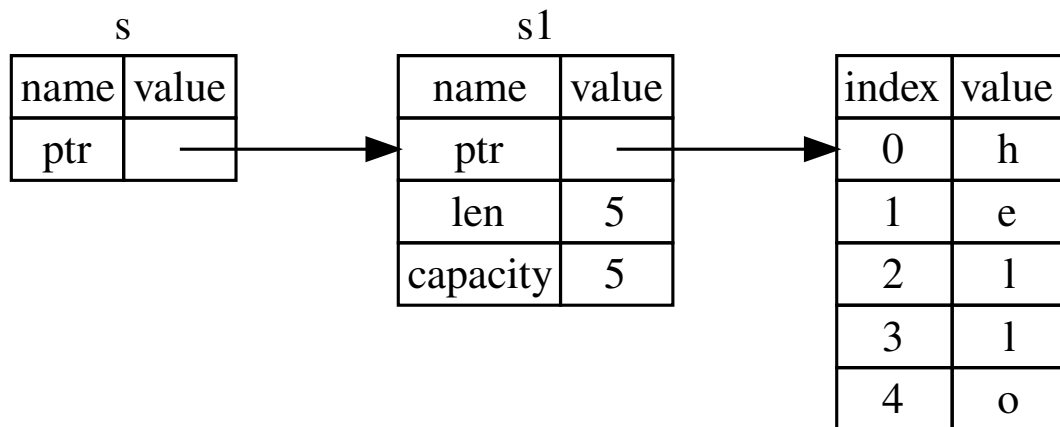


- ```
fn main() {
 let s1 = String::from("hello");

 let len = calculate_length(&s1);

 println!("The length of '{s1}' is {len}.");
}

fn calculate_length(s: &String) -> usize {
 s.len()
}
```



# Ownership – References



- Reference does not take ownership!!
- It is a pointer to the variable
- Creating a reference (the pointer) is called **borrowing**!
- As the pointer is of a known (rather small) size, it can be dropped without affecting the original value it points to
- Thus, the problem is solved 😊

# Ownership – Borrowing



- Borrowing means no modification...
- This won't work:
- ```
fn main() {  
    let s = String::from("hello");  
  
    change(&s);  
}  
  
fn change(some_string: &String) {  
    some_string.push_str(", world");  
}
```

Ownership – Mutable References



- Is there any solution? Yes!!
- **Mutable references!**
- ```
fn main() {
 let mut s = String::from("hello");

 change(&mut s);
}

fn change(some_string: &mut String) {
 some_string.push_str(", world");
}
```

# Ownership – Mutable References



- Only one, single **one mutable reference** can be in charge!
- This will **NOT** compile:

- ```
let mut s = String::from("hello");
```

```
let r1 = &mut s;
```

```
let r2 = &mut s;
```

```
println!("{}", {}, r1, r2);
```



Ownership – Mutable and Immutable Refs



- We can have as many as required of **immutable** references
- We **cannot combine them with mutable** ones
- `let mut s = String::from("hello");`

```
let r1 = &s; // no problem
```

```
let r2 = &s; // no problem
```

```
let r3 = &mut s; // BIG PROBLEM
```

```
println!("{}", {}, and {}", r1, r2, r3);
```



Ownership – Mutable and Immutable Refs



- Reference is valid till its last use...

- Thus, this will work:

- `let mut s = String::from("hello");`

```
let r1 = &s; // no problem
```

```
let r2 = &s; // no problem
```

```
println!("{r1} and {r2}");
```

```
// variables r1 and r2 will not be used after this point
```

```
let r3 = &mut s; // no problem then
```

```
println!("{r3}");
```

Ownership – Dangling Reference



- Reference to non-existing object – the memory is used by something else or not at all
 - Typical error C/C++ and elsewhere
- E.g. (it will not compile in rust)

```
fn main() {  
    let reference_to_nothing = dangle();  
}  
  
fn dangle() -> &String {  
    let s = String::from("hello");  
    &s  
}
```



Ownership – Dangling Reference



- It is closely coupled with variable lifetime
 - Do you remember IPP? 😊
- The solution may be to return the value directly
- ```
fn no_dangle() -> String {
 let s = String::from("hello");

 s
}
```

# Ownership – Rules for References



- At any given time, you can have either one mutable reference or any number of immutable references.
- References must always be valid.



-

# Slices – Finding the First Word



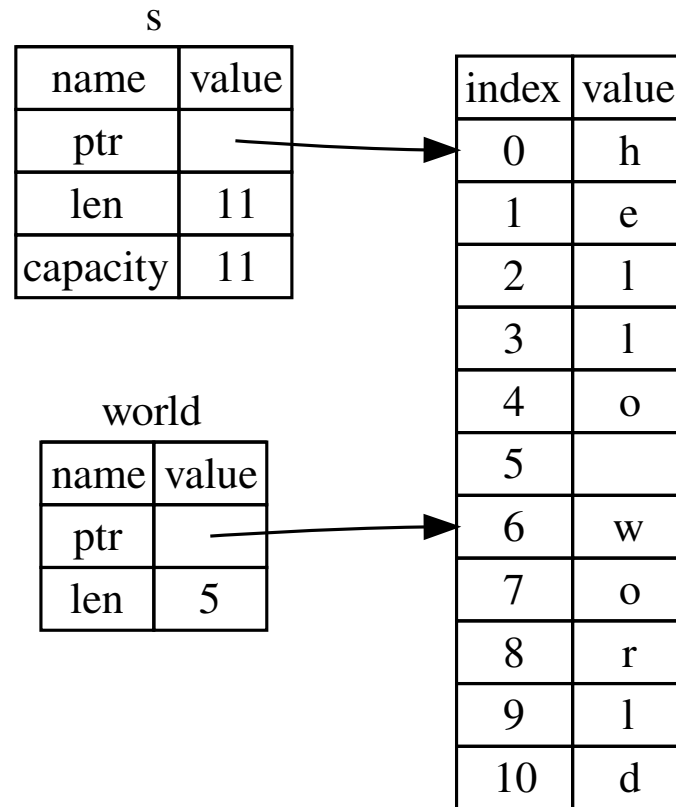
- ```
fn first_word(s: &String) -> usize {  
    let bytes = s.as_bytes();  
    for (i, &item) in bytes.iter().enumerate() {  
        if item == b' ' {  
            return i;  
        }  
    }  
    s.len()  
}
```
- Too clumsy, though...
- I'd rather have a substring, wouldn't you?

String Slices



- A string slice is a reference to part of a String:
- **`let s = String::from("hello world");`**

```
let hello = &s[0..5];  
let world = &s[6..11];
```



String Slices – Usage



- `let s = String::from("hello");`

```
let slice = &s[0..2];  
let slice = &s[..2];
```

- `// -----`

- `let s = String::from("hello");`

```
let len = s.len();
```

```
let slice = &s[3..len];  
let slice = &s[3..];
```

- `let s = String::from("hello");`

```
let len = s.len();
```

```
let slice = &s[0..len];  
let slice = &s[..];
```

Slices – Task Revised



- ```
fn first_word(s: &String) -> &str
{
 let bytes = s.as_bytes();

 for (i, &item) in
 bytes.iter().enumerate() {
 if item == b' ' {
 return &s[0..i];
 }
 }

 &s[..]
}
```

- Slice is **not** of a type String!
- String literal, e.g. “Hi!”, is of a type **&str**
- Values of a type String can be passed to parameters of a type &str (string slice) directly, thus...

# Slices – Task Revised



- ```
fn first_word(s: &str) -> &str
{
    let bytes = s.as_bytes();

    for (i, &item) in
        bytes.iter().enumerate() {
            if item == b' ' {
                return &s[0..i];
            }
        }

    &s[..]
}
```

- More Rusteous 😊
- Simpler usage
(see next slide)
- Less Haskellish more
Pythonish 🤔

Slices – Usage of Revised Function



```
• fn main() {  
    let my_string = String::from("hello world");  
  
    // `first_word` works on slices of `String`s, whether partial or whole  
    let word = first_word(&my_string[0..6]);  
    let word = first_word(&my_string[..]);  
    // `first_word` also works on references to `String`s, which are equivalent  
    // to whole slices of `String`s  
    let word = first_word(&my_string);  
  
    let my_string_literal = "hello world";  
  
    // `first_word` works on slices of string literals, whether partial or whole  
    let word = first_word(&my_string_literal[0..6]);  
    let word = first_word(&my_string_literal[..]);  
  
    // Because string literals are string slices already,  
    // this works too, without the slice syntax!  
    let word = first_word(my_string_literal);  
}
```

Other Slices



- What about arrays??
- Yes!
- ```
let a = [1, 2, 3, 4, 5];
let slice = &a[1..3];
assert_eq!(slice, &[2, 3]);
```

# Data Structures



- Haskell:
  - ```
data User = User {  
    active :: Boolean,  
    username :: String,  
    email :: String,  
    signInCount :: integer  
}
```
- Rust:
 - ```
struct User {
 active: bool,
 username: String, // we cannot use &str - see Lifetimes
 email: String,
 sign_in_count: u64,
}
```

# Structure Creation



- ```
fn main() {  
    let mut user1 = User {  
        active: true,  
        username: String ::  
            from("someusername123"),  
        email: String ::  
            from("someone@example.com"),  
        sign_in_count: 1,  
    };  
  
    user1.email = String ::  
        from("anotheremail@example.com");  
}
```

- ```
fn build_user(
 email: String,
 username: String) -> User {
 User {
 active: true,
 username,
 email,
 sign_in_count: 1,
 }
}
```

# Structure Creation by Copying



- ```
fn main() {  
    // --snip--  
  
    let user2 = User {  
        active: user1.active,  
        username: user1.username,  
        email: String ::  
            from("another@example.com"),  
        sign_in_count:  
            user1.sign_in_count,  
    };  
}
```

- ```
fn main() {
 // --snip--

 let user2 = User {
 email: String ::
 from("another@example.com"),
 ..user1
 };
}
```

# Structure Creation – Haskell Style



- ```
struct Color(i32, i32, i32);  
struct Point(i32, i32, i32);  
  
fn main() {  
    let black = Color(0, 0, 0);  
    let origin = Point(0, 0, 0);  
}
```

Unlike Structs



- Data unit () – of course (Haskellish 😊)
- **struct AlwaysEqual;**

```
fn main() {  
    let subject = AlwaysEqual;  
}
```

- Equal with any other structure instance

Debugging Structures



- Do you remember “deriving” from Haskell?

- ```
#[derive(Debug)]
struct Rectangle {
 width: u32,
 height: u32,
}

fn main() {
 let scale = 2;
 let rect1 = Rectangle {
 width: dbg!(30 * scale),
 Height: 50,
 };

 dbg!(&rect1);
}
```



# Debugging Structures – Output



- `./target/release/struct_dbg`  
`[src/main.rs:11:16] 30 * scale = 60`  
`[src/main.rs:15:5] &rect1 = Rectangle {`  
    `width: 60,`  
    `height: 50,`  
`}`
- We'll see more for `derive`

# Methods



- ```
#[derive(Debug)]  
struct Rectangle {  
    width: u32,  
    height: u32,  
}  
  
impl Rectangle {  
    fn area(&self) -> u32 {  
        self.width * self.height  
    }  
}
```

- Haskell style... 😊
SELF character

- ```
fn main() {
 let rect1 = Rectangle {
 width: 30,
 height: 50,
 };

 println!(
 "The area of the
 rectangle is {}
 square pixels.",
 rect1.area()
);
}
```

# Methods – Naming Convention



- ```
impl Rectangle {  
    fn width(&self) -> bool {  
        self.width > 0  
    }  
}  
  
fn main() {  
    let rect1 = Rectangle {  
        width: 30,  
        height: 50,  
    };  
  
    if rect1.width() {  
        println!("The rectangle has a nonzero width; it is {}",  
            rect1.width);  
    }  
}
```

Methods – More Parameters



- **impl Rectangle {**
 fn area(&self) -> u32 {
 self.width * self.height
 }

 fn can_hold(&self, other: &Rectangle) -> bool {
 self.width > other.width && self.height > other.height
 }
}
- Multiple **impl** blocks allowed too

Associated Functions



- Used for constructors, usually
- ```
impl Rectangle {
 fn square(size: u32) -> Self {
 Self {
 width: size,
 height: size,
 }
 }
}
```
- ```
let sq = Rectangle::square(3);
```

Enums



- **enum IpAddrKind {
 V4,
 V6,
}**
- **let four = IpAddrKind::V4;
let six = IpAddrKind::V6;**
- Pretty straightforward

Typed Enums



- ```
enum IpAddr {
 V4(String),
 V6(String),
}
let home = IpAddr::V4(String::from("127.0.0.1"));
let loopback = IpAddr::V6(String::from("::1"));
```
- ```
enum IpAddr {  
    V4(u8, u8, u8, u8),  
    V6(String),  
}  
let home = IpAddr::V4(127, 0, 0, 1);  
let loopback = IpAddr::V6(String::from("::1"));
```

Type Enums



- ```
enum Message {
 Quit,
 Move { x: i32, y: i32 },
 Write(String),
 ChangeColor(i32, i32, i32),
}
```
- Similar to Haskell **data** variants
- ```
let m = Message::Write(String::from("hello"));  
m.call();
```


Option Enum



- Haskell equivalent is Maybe type 🤔
- **enum Option<T> {
 None,
 Some(T),
}**
- Even type variable is present 😊
- **let some_number = Some(5);
let some_char = Some('e');
let absent_number: Option<i32> = None;**

Match – Pattern Matching



- The same as **case** in Haskell, just different syntax

- ```
enum Coin {
 Penny,
 Nickel,
 Dime,
 Quarter,
}
fn value_in_cents(coin: Coin) -> u8 {
 match coin {
 Coin::Penny => 1,
 Coin::Nickel => 5,
 Coin::Dime => 10,
 Coin::Quarter => 25,
 }
}
```

# Match over Parameters



- ```
#[derive(Debug)]  
    // so we can inspect  
    // the state in a minute  
enum UsState {  
    Alabama,  
    Alaska,  
    // --snip--  
}
```
- ```
enum Coin {
 Penny,
 Nickel,
 Dime,
 Quarter(UsState),
}
```
- ```
fn value_in_cents(coin: Coin)  
    -> u8 {  
    match coin {  
        Coin::Penny => 1,  
        Coin::Nickel => 5,  
        Coin::Dime => 10,  
        Coin::Quarter(state) => {  
            println!("State quarter  
                from {state:?}!");  
            25  
        }  
    }  
}
```

Matching Option Type



- ```
fn plus_one(x: Option<i32>) -> Option<i32> {
 match x {
 None => None,
 Some(i) => Some(i + 1),
 }
}
```
- ```
let five = Some(5);  
let six  = plus_one(five);  
let none = plus_one(None);
```
- Note: you **MUST** provide **all cases**, not Haskellish

Match – “All” Catcher



- ```
let dice_roll = 9;
match dice_roll {
 3 => add_fancy_hat(),
 7 => remove_fancy_hat(),
 _ => (), // catches remaining, () for nothing
}
```
- ```
fn add_fancy_hat() { ... }
```
- ```
fn remove_fancy_hat() { ... }
```

# Combining if, let, else



- ```
let config_max = Some(3u8);  
match config_max {  
    Some(max) =>  
        println!("The maximum is  
        configured to be {max}"),  
    _ => (),  
}
```

- ```
let config_max = Some(3u8);
if let Some(max) = config_max {
 println!("The maximum is
 configured to be {max}");
}
```

- ```
let mut count = 0;  
match coin {  
    Coin::Quarter(state) =>  
        println!("State quarter  
        from {state:?}!"),  
    _ => count += 1,  
}
```

- ```
let mut count = 0;
if
 let Coin::Quarter(state) = coin {
 println!("State quarter
 from {state:?}!");
 } else {
 count += 1;
 }
```

# Combining if, let, else



- ```
fn describe_state_quarter(coin: Coin) -> Option<String> {  
    let state = if let Coin::Quarter(state) = coin {  
        State  
    } else {  
        return None;  
    };  
  
    if state.existed_in(1900) {  
        Some(format!("{state:?} is pretty old, for America!"))  
    } else {  
        Some(format!("{state:?} is relatively new."))  
    }  
}
```

Combining if, let, else



- ```
fn describe_state_quarter(coin: Coin) -> Option<String> {
 let Coin::Quarter(state) = coin else {
 return None;
 };

 if state.existed_in(1900) {
 Some(format!("{state:?} is pretty old, for America!"))
 } else {
 Some(format!("{state:?} is relatively new."))
 }
}
```



# Packages, Crates, Modules, Paths



- **Packages:** A Cargo feature that lets you build, test, and share *crates*
- **Crates:** A *tree of modules* that produces a **library or executable**
- **Modules** and **use:** Let you control the organization, scope, and privacy of *paths*
- **Paths:** A way of naming an item, such as a struct, function, or module

# Crates (přepřavky)



- **Binary** crate – project providing an executable
- **Library** crate – project providing library/module code, can't be executed on its own
- Cargo.toml

```
[dependencies]
rand = "0.8.5"
```
- Running build will import and build with dependencies
  - See doc.
- `cargo add regex`

# Package



- A **package** is a bundle of one or more crates that provides a set of functionality.
- A package contains a **Cargo.toml** file that describes how to build those crates.
- A package can contain **as many binary crates as you like**, but **at most only one library crate**.
- A package must contain **at least one crate**, whether that's a library or binary crate.

# Module Search



- Start from the crate root: `src/lib.rs` or `src/main.rs`
- Declaring modules: In the crate root file, you can declare new modules; say you declare a “garden” module with `mod garden`;
  - Inline, within curly brackets that replace the semicolon following `mod garden`
  - In the file `src/garden.rs`
  - In the file `src/garden/mod.rs`
- Declaring submodules: In any file other than the crate root, you can declare submodules. For example, you might declare `mod vegetables`; in `src/garden.rs`.
  - Inline, directly following `mod vegetables`, within curly brackets instead of the semicolon
  - In the file `src/garden/vegetables.rs`
  - In the file `src/garden/vegetables/mod.rs`

# Modules Search – continued



- Paths to code in modules:
  - for example, an Asparagus type in the garden vegetables module would be found at `crate::garden::vegetables::Asparagus`
- Private vs. public: Code within a module is private from its parent modules by default.
  - public module, declare it with `pub mod` instead of `mod`
- The use keyword:
  - `crate::garden::vegetables::Asparagus`
  - `use crate::garden::vegetables::Asparagus;`
  - ...
  - `Asparagus`

# Modules – Tree



- backyard
  - ├── Cargo.lock
  - ├── Cargo.toml
  - └── src
    - ├── garden
      - └── vegetables.rs
    - ├── garden.rs
    - └── main.rs

# Nested Modules – e.g. lib.rs



- ```
mod front_of_house {  
    mod hosting {  
        fn add_to_waitlist() {}  
  
        fn seat_at_table() {}  
    }  
  
    mod serving {  
        fn take_order() {}  
  
        fn serve_order() {}  
  
        fn take_payment() {}  
    }  
}
```

Referring Modules



- Absolute path: starts with **crate**
- Relative path: starts from actual module, uses **self**, **super**
- lib.rs:

```
mod front_of_house {  
    mod hosting {  
        fn add_to_waitlist() {}  
    }  
}  
  
pub fn eat_at_restaurant() {  
    // Absolute path  
    crate::front_of_house::hosting::add_to_waitlist();  
    // Relative path  
    front_of_house::hosting::add_to_waitlist();  
}
```



Referring Modules – Public



- lib.rs revised
- ```
mod front_of_house {
 pub mod hosting {
 pub fn add_to_waitlist() {}
 }
}
```

```
pub fn eat_at_restaurant() {
 // Absolute path
 crate::front_of_house::hosting::add_to_waitlist();
 // Relative path
 front_of_house::hosting::add_to_waitlist();
}
```

# Referring Modules – Super



- lib.rs:

- **fn deliver\_order() {}**

```
mod back_of_house {
 fn fix_incorrect_order() {
 cook_order();
 super::deliver_order();
 }

 fn cook_order() {}
}
```

# Structs Public



```
• mod back_of_house {
 pub struct Breakfast {
 pub toast: String,
 seasonal_fruit: String, // private attribute
 }
 impl Breakfast {
 pub fn summer(toast: &str) -> Breakfast {
 Breakfast {
 toast: String::from(toast),
 seasonal_fruit: String::from("peaches"),
 }
 }
 }
}

pub fn eat_at_restaurant() {
 // Order a breakfast in the summer with Rye toast
 let mut meal = back_of_house::Breakfast::summer("Rye");
 // Change our mind about what bread we'd like
 meal.toast = String::from("Wheat");
 println!("I'd like {} toast please", meal.toast);
 // The next line won't compile if we uncomment it; we're not allowed
 // to see or modify the seasonal fruit that comes with the meal
 // meal.seasonal_fruit = String::from("blueberries");
}
```

# Enum (All) Public



- lib.rs:
- ```
mod back_of_house {  
    pub enum Appetizer {  
        Soup,           // public by default  
        Salad,          // public by default  
    }  
}  
  
pub fn eat_at_restaurant() {  
    let order1 = back_of_house::Appetizer::Soup;  
    let order2 = back_of_house::Appetizer::Salad;  
}
```

Using use



- ```
mod front_of_house {
 pub mod hosting {
 pub fn add_to_waitlist() {}
 }
}
```

```
use crate::front_of_house::hosting;
```

```
pub fn eat_at_restaurant() {
 hosting::add_to_waitlist();
}
```

# Using use



- ```
mod front_of_house {  
    pub mod hosting {  
        pub fn add_to_waitlist() {}  
    }  
}
```

use crate::front_of_house::hosting;

```
mod customer {  
  
    pub fn eat_at_restaurant() {  
        hosting::add_to_waitlist();  
    }  
}
```



Using use



- ```
mod front_of_house {
 pub mod hosting {
 pub fn add_to_waitlist() {}
 }
}
```

```
mod customer {
 use crate::front_of_house::hosting;
 pub fn eat_at_restaurant() {
 hosting::add_to_waitlist();
 }
}
```

# Using use



- ```
mod front_of_house {  
    pub mod hosting {  
        pub fn add_to_waitlist() {}  
    }  
}
```

use crate::front_of_house::hosting;

```
mod customer {  
  
    pub fn eat_at_restaurant() {  
        super.hosting::add_to_waitlist();  
    }  
}
```


Idiomatic use



- ```
mod front_of_house {
 pub mod hosting {
 pub fn add_to_waitlist() {}
 }
}
```

```
use crate::front_of_house::hosting::add_to_waitlist;
```

```
pub fn eat_at_restaurant() {
 add_to_waitlist();
}
```

# Idiomatic use



- `use std::collections::HashMap;`

```
fn main() {
 let mut map = HashMap::new();
 map.insert(1, 2);
}
```

- `use std::fmt;`  
`use std::io;`

```
fn function1() →
 fmt::Result {

 // --snip--
}
```

```
fn function2() ->
 io::Result<()> {

 // --snip--
}
```

# Using use and as



- `use std::fmt::Result;`  
`use std::io::Result as IoResult;`

```
fn function1() -> Result {
 // --snip--
}
```

```
fn function2() -> IoResult<()> {
 // --snip--
}
```

# Re-exporting with pub and use



- ```
mod front_of_house {  
    pub mod hosting {  
        pub fn add_to_waitlist() {}  
    }  
}
```

```
pub use crate::front_of_house::hosting;  
// available for any code to use from a new scope
```

```
pub fn eat_at_restaurant() {  
    hosting::add_to_waitlist();  
}
```

Using External Packages



- Toml file, or cargo add ...

- E.g. **use std::io;**

- Still qualified usage:

```
io::stdin()  
    .read_line(&mut guess)  
    .expect("Failed to read line");
```

- **use std::cmp::Ordering; => use std::{cmp::Ordering, io};**
use std::io;
- **use std::collections::*;**

Several Files



- `src/lib.rs`
 - ```
mod front_of_house;
pub use crate::front_of_house::hosting;
pub fn eat_at_restaurant() {
 hosting::add_to_waitlist();
}
```
- `src/front_of_house.rs`
  - ```
pub mod hosting {  
    pub fn add_to_waitlist() {}  
}
```

Several Files Complicated



- `src/lib.rs`
 - `mod front_of_house;`
`pub use crate::front_of_house::hosting;`
`pub fn eat_at_restaurant() {`
 `hosting::add_to_waitlist();`
`}`
- `src/front_of_house.rs`
 - `pub mod hosting;`
- `src/front_of_house/hosting.rs`
 - `pub fn add_to_waitlist() {}`

Vectors



- Similar to C++ (and others)
- **let v: Vec<i32> = Vec::new();**
- **let v = vec![1, 2, 3];**
- **let mut v = Vec::new();**
 v.push(5);
 v.push(6);
 v.push(7);
 v.push(8);

Vectors – Reading



- `let v = vec![1, 2, 3, 4, 5];`

```
let third: &i32 = &v[2];  
println!("The third element is {third}");
```

```
let third: Option<&i32> = v.get(2);  
match third {  
    Some(third) => println!("The third element is {third}"),  
    None => println!("There is no third element."),  
}
```

Iterating Over Vector Values



- ```
let v = vec![100, 32, 57];
for i in &v {
 println!("{i}");
}
```
- ```
let mut v = vec![100, 32, 57];  
for i in &mut v {  
    *i += 50;  
}
```
- Note: dropping a Vector drop its elements!

Enum Enables to Store Multiple Types



- ```
enum SpreadsheetCell { // general data without recursion
 Int(i32), // in Haskell
 Float(f64),
 Text(String),
}

let row = vec![
 SpreadsheetCell::Int(3),
 SpreadsheetCell::Text(String::from("blue")),
 SpreadsheetCell::Float(10.12),
];
```

# Strings



- Built-in – string slice (above)
- String is a part of the standard library, stores UTF-8 strings
- **let mut s = String::new();**
- **let data = "initial contents";  
let s = data.to\_string();**
- **let s = "initial contents".to\_string();**
- **let s = String::from("initial contents");**
- **let hello = String::from("Krásný večer!");**

# String Change



- Takes slices, thus this compiles:
- ```
let mut s1 = String::from("foo");  
let s2 = "bar";  
s1.push_str(s2);  
println!("s2 is {s2}");
```
- ```
let mut s = String::from("lo");
s.push('l');
```
-

# String Concatenation



- `let s1 = String::from("Hello, ");`  
`let s2 = String::from("world!");`  
`let s3 = s1 + &s2;`  
`// note s1 has been moved here and can no longer be used`
- `fn add(self, s: &str) -> String { ... }`
- This allows to build your own add

# String Concatenation



- ```
let s1 = String::from("tic");  
let s2 = String::from("tac");  
let s3 = String::from("toe");  
  
let s = s1 + "-" + &s2 + "-" + &s3;
```
- ```
let s1 = String::from("tic");
let s2 = String::from("tac");
let s3 = String::from("toe");

let s = format!("{s1}-{s2}-{s3}");
```

# String Indexing



- Not allowed due to UTF-8
- Not even via slicing
- Strings are not to simple as they seem to be...
- ```
for c in "3д".chars() {  
    println!("{c}");    // 2 characters printed  
}
```
- ```
for b in "3д".bytes() {
 println!("{b}"); // 4 bytes printed
}
```



# Hash Maps



- **use `std::collections::HashMap`;**

```
let mut scores = HashMap::new();
```

```
scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);
```

- **let `team_name = String::from("Blue")`;  
let `score = scores.get(&team_name).copied().unwrap_or(0)`;**

Option<&V> copied to Option<i32>, unwrap\_or set to zero if...

# Hash Maps



- **use `std::collections::HashMap`;**

```
let mut scores = HashMap::new();
```

```
scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);
```

```
for (key, value) in &scores {
 println!("{key}: {value}");
}
```

- Yellow: 50  
Blue: 10

# Ownership in Hash Maps



- For types that implement the **Copy** trait, like **i32**, the values are **copied** into the hash map.
- For **owned** values like **String**, the values will be **moved** and the hash map will be the owner of those values.

# Overwriting in Hash Map



- **use `std::collections::HashMap`;**

```
let mut scores = HashMap::new();
```

```
scores.insert(String::from("Blue"), 10);
```

```
scores.insert(String::from("Blue"), 25);
```

```
// only this will be present
```

```
println!("{scores:?}");
```

# Keep Old Values in Hash Map



- **use `std::collections::HashMap`;**

```
let mut scores = HashMap::new();
scores.insert(String::from("Blue"), 10);
```

```
scores.entry(String::from("Yellow")).or_insert(50);
scores.entry(String::from("Blue")).or_insert(50);
```

```
println!("{scores:?}");
```

# Update An Old Value in Hash Map



- **use `std::collections::HashMap`;**

```
let text = "hello world wonderful world";
```

```
let mut map = HashMap::new();
```

```
for word in text.split_whitespace() {
 let count = map.entry(word).or_insert(0);
 *count += 1;
}
```

```
println!("{map:?}");
```

- **`{"world": 2, "hello": 1, "wonderful": 1}`**

# Erroneous Situations



- Panic!

- **fn main() {  
    panic!("crash and burn");  
}**
  - \$ cargo run  
    Compiling panic v0.1.0 (file:///projects/panic)  
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.25s  
    Running `target/debug/panic`  
thread 'main' panicked at src/main.rs:2:5:  
crash and burn  
note: run with `RUST\_BACKTRACE=1` environment variable to display a  
backtrace

- **RUST\_BACKTRACE=1 cargo run**

# We Want to Recover



- ```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```
- ```
use std::fs::File;

fn main() {
 let greeting_file_result = File::open("hello.txt");

 let greeting_file = match greeting_file_result {
 Ok(file) => file,
 Err(error) => panic!("Problem opening the file: {error:?}"),
 };
}
```



# Create File If Missing



- ```
use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let greeting_file_result = File::open("hello.txt");

    let greeting_file = match greeting_file_result {
        Ok(file) => file,
        Err(error) => match error.kind() {
            ErrorKind::NotFound => match File::create("hello.txt") {
                Ok(fc) => fc,
                Err(e) => panic!("Problem creating the file: {e:?}"),
            },
            _ => {
                panic!("Problem opening the file: {error:?}");
            }
        },
    };
}
```

Alternative Handling of the Same Problem



- ```
use std::fs::File;
use std::io::ErrorKind;

fn main() {
 let greeting_file
 = File::open("hello.txt").unwrap_or_else(|error| {
 if error.kind() == ErrorKind::NotFound {
 File::create("hello.txt").unwrap_or_else(|error| {
 panic!("Problem creating the file: {error:?}");
 })
 } else {
 panic!("Problem opening the file: {error:?}");
 }
 });
}
```

# Direct Panic



- **use std::fs::File;**

```
fn main() {
 let greeting_file = File::open("hello.txt").unwrap();
}
```

- **use std::fs::File;**

```
fn main() {
 let greeting_file = File::open("hello.txt")
 .expect("hello.txt should be included in the project");
}
```

# Read File with Error Propagation



- ```
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let username_file_result = File::open("hello.txt");

    let mut username_file = match username_file_result {
        Ok(file) => file,
        Err(e) => return Err(e),
    };

    let mut username = String::new();

    match username_file.read_to_string(&mut username) {
        Ok(_) => Ok(username),
        Err(e) => Err(e),
    }
}
```

A Shortcut for Error Propagation



- **use std::fs::File;
use std::io::{self, Read};**

```
fn read_username_from_file() -> Result<String, io::Error> {  
    let mut username_file = File::open("hello.txt")?;  
    let mut username = String::new();  
    username_file.read_to_string(&mut username)?;  
    Ok(username)  
}
```

Even Shorter...



- **use std::fs::File;**
use std::io::{self, Read};

```
fn read_username_from_file() -> Result<String, io::Error> {  
    let mut username = String::new();  
  
    File::open("hello.txt").?.read_to_string(&mut username)?;  
  
    Ok(username)  
}
```

Extra Short!



- **use std::fs;
use std::io;**

```
fn read_username_from_file() -> Result<String, io::Error> {  
    fs::read_to_string("hello.txt")  
}
```

It Is Strongly Typed, though



- **use std::fs::File;**

```
fn main() {  
    let greeting_file = File::open("hello.txt"?;  
}
```

- ... will not compile, while the following will:

- **use std::error::Error;**
use std::fs::File;

```
fn main() -> Result<(), Box<dyn Error>> {  
    let greeting_file = File::open("hello.txt"?;  
  
    Ok::<(), Box<dyn Error>>()  
}
```


Generic Types, ...



- Do not copy the code, make functions!
 - In Haskell, especially there!

```
• fn main() {  
    let number_list = vec![34, 50, 25, 100, 65];  
    let mut largest = &number_list[0];  
    for number in &number_list {  
        if number > largest {  
            largest = number;  
        }  
    }  
    println!("The largest number is {largest}");  
  
    let number_list = vec![102, 34, 6000, 89, 54, 2, 43, 8];  
    let mut largest = &number_list[0];  
    for number in &number_list {  
        if number > largest {  
            largest = number;  
        }  
    }  
    println!("The largest number is {largest}");  
}
```

Making a Function Is a Good Approach



```
• fn largest(list: &[i32]) -> &i32 {  
    let mut largest = &list[0];  
    for item in list {  
        if item > largest {  
            largest = item;  
        }  
    }  
    largest  
}  
  
fn main() {  
    let number_list = vec![34, 50, 25, 100, 65];  
    let result = largest(&number_list);  
    println!("The largest number is {result}");  
  
    let number_list = vec![102, 34, 6000, 89, 54, 2, 43, 8];  
    let result = largest(&number_list);  
    println!("The largest number is {result}");  
}
```

It Seems To Be a Bit Clumsy



- ```
fn largest_i32(list: &[i32]) -> &i32 {
 ...
}

fn largest_char(list: &[char]) -> &char {
 ...
}

fn main() {
 let number_list = vec![34, 50, 25, 100, 65];
 let result = largest_i32(&number_list);
 println!("The largest number is {result}");

 let char_list = vec!['y', 'm', 'a', 'q'];
 let result = largest_char(&char_list);
 println!("The largest char is {result}");
}
```

# Trying a Haskell Approach...



```
• fn largest<T>(list: &[amp;T]) -> &T {
 let mut largest = &list[0];
 for item in list {
 if item > largest {
 largest = item;
 }
 }
 largest
}

fn main() {
 let number_list = vec![34, 50, 25, 100, 65];
 let result = largest(&number_list);
 println!("The largest number is {result}");

 let char_list = vec!['y', 'm', 'a', 'q'];
 let result = largest(&char_list);
 println!("The largest char is {result}");
}
```



# Parametric Structures – Haskell Like



- ```
struct Point<T> {  
    x: T,  
    y: T,  
}
```



```
fn main() {  
    let integer = Point { x: 5, y: 10 };  
    let float = Point { x: 1.0, y: 4.0 };  
}
```
- You cannot mix types!!
- This is strongly typed language!!

Do You Need Different Types?



- **struct Point<T, U> {
 x: T,
 y: U,
}**

**fn main() {
 let both_integer = Point { x: 5, y: 10 };
 let both_float = Point { x: 1.0, y: 4.0 };
 let integer_and_float = Point { x: 5, y: 4.0 };
}**

The Same for Enums



- **enum Option<T> {
 Some(T),
 None,
}**
- **enum Result<T, E> {
 Ok(T),
 Err(E),
}**

Defining Methods



```
• struct Point<T> {  
    x: T,  
    y: T,  
}  
  
impl<T> Point<T> {  
    fn x(&self) -> &T {  
        &self.x  
    }  
}  
  
fn main() {  
    let p = Point { x: 5, y: 10 };  
  
    println!("p.x = {}", p.x());  
}
```


Type Specific Methods



- `impl Point<f32> {
 fn distance_from_origin(&self) -> f32 {
 (self.x.powi(2) + self.y.powi(2)).sqrt()
 }
}`

Mixing the Stuff



```
• struct Point<X1, Y1> {  
    x: X1,  
    y: Y1,  
}  
  
impl<X1, Y1> Point<X1, Y1> {  
    fn mixup<X2, Y2>(self, other: Point<X2, Y2>) -> Point<X1, Y2> {  
        Point {  
            x: self.x,  
            y: other.y,  
        }  
    }  
}  
  
fn main() {  
    let p1 = Point { x: 5, y: 10.4 };  
    let p2 = Point { x: "Hello", y: 'c' };  
  
    let p3 = p1.mixup(p2);  
  
    println!("p3.x = {}, p3.y = {}", p3.x, p3.y);  
}
```

Traits



- A trait defines the functionality a particular type has and can share with other types.
- Haskell type classes are the same.
- Interfaces in some OO languages too.
- e.g.

```
– pub trait Summary {  
    fn summarize(&self) -> String;  
}
```

Implementing Traits 1



- ```
pub struct NewsArticle {
 pub headline: String,
 pub location: String,
 pub author: String,
 pub content: String,
}
```

```
impl Summary for NewsArticle {
 fn summarize(&self) -> String {
 format!("{}, by {} ({}))", self.headline, self.author,
 self.location)
 }
}
```

# Implementing Traits 2



- ```
pub struct SocialPost {  
    pub username: String,  
    pub content: String,  
    pub reply: bool,  
    pub repost: bool,  
}  
  
impl Summary for SocialPost {  
    fn summarize(&self) -> String {  
        format!("{}: {}", self.username, self.content)  
    }  
}
```

Using Traits



- `use aggregator::{SocialPost, Summary};`

```
fn main() {  
    let post = SocialPost {  
        username: String::from("horse_ebooks"),  
        content: String::from(  
            "of course, as you probably already know, people",  
        ),  
        reply: false,  
        repost: false,  
    };  
  
    println!("1 new social post: {}", post.summarize());  
}
```

Default Implementation



- ```
pub trait Summary {
 fn summarize(&self) -> String {
 String::from("(Read more...)")
 }
}
```
- ```
impl Summary for NewsArticle {} // void instance
```

```
let article = NewArticle { ... }
```

```
println!("New article available! {}", article.summarize());
```

Defaults Can Use Other Methods



- ```
pub trait Summary {
 fn summarize_author(&self) -> String;

 fn summarize(&self) -> String {
 format!("(Read more from {})...", self.summarize_author())
 }
}
```
- ```
impl Summary for SocialPost {  
    fn summarize_author(&self) -> String {  
        format!("@{}", self.username)  
    }  
}
```


Traits as Parameters



- **pub fn notify(item: &impl Summary) {
 println!("Breaking news! {}", item.summarize());
}**
- ... is a syntax sugar for ...
- **pub fn notify<T: Summary>(item: &T) {
 println!("Breaking news! {}", item.summarize());
}**

More Traits More Adidas? 😊



- Sugar used:
- **pub fn notify(item: &(impl Summary + Display)) { ... }**
- De-sugared:
- **pub fn notify<T: Summary + Display>(item: &T) { ... }**

Using **where** Clauses



- A bit Haskellish, isn't it? 🤔 😊
- De-sugared:
- **fn some_function<T: Display + Clone, U: Clone + Debug>(t: &T, u: &U) -> i32 { ... }**
- Using **where** clause:
- **fn some_function<T, U>(t: &T, u: &U) -> i32
where
 T: Display + Clone,
 U: Clone + Debug,
 { ... }**

Lifetimes



- Yes, again “useless” “buzzword” from IPP
- *Lifetimes are another kind of generic that we’ve already been using. Rather than ensuring that a type has the behavior we want, lifetimes ensure that references are valid as long as we need them to be.*

```
• fn main() {  
    let r;  
    {  
        let x = 5;  
        r = &x;  
    }  
    println!("r: {r}");  
}
```

```
fn main () {  
    let x = 5;  
  
    let r= &x;  
  
    println!("r: {r}");  
}
```

Example



- ```
fn main() {
 let string1 = String::from("abcd");
 let string2 = "xyz";
 let result = longest(string1.as_str(), string2);
 println!("The longest string is {result}");
}
```
- ```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() { x } else { y }  
}
```

Lifetimes



- **`&i32` `// a reference`**
- **`&'a i32` `// a reference with an explicit lifetime`**
- **`&'a mut i32` `// a mutable reference with an explicit lifetime`**
- The function signature now tells Rust that for some lifetime 'a, the function takes two parameters, both of which are string slices that live at least as long as lifetime 'a.
- The function signature also tells Rust that the string slice returned from the function will live at least as long as lifetime 'a.
- In practice, it means that the lifetime of the reference returned by the longest function is the same as the smaller of the lifetimes of the values referred to by the function arguments.
- These relationships are what we want Rust to use when analyzing this code.

Lifetimes



- When returning a reference from a function, the lifetime parameter for the return type needs to match the lifetime parameter for one of the parameters.
- *If the reference returned does not refer to one of the parameters, it must refer to a value created within this function.*
However, this would be a dangling reference because the value will go out of scope at the end of the function.

Lifetime in Struct



- ```
struct ImportantExcerpt<'a> {
 part: &'a str,
}

fn main() {
 let novel
 = String::from("Call me Ishmael. Some years ago...");
 let first_sentence = novel.split('.').next().unwrap();
 let i = ImportantExcerpt {
 part: first_sentence,
 };
}
```



## ... Brings Lifetime to Methods



- ```
impl<'a> ImportantExcerpt<'a> {  
    fn level(&self) -> i32 {  
        3  
    }  
}
```
- ```
impl<'a> ImportantExcerpt<'a> {
 fn announce_and_return_part(&self, announcement: &str) -> &str {
 println!("Attention please: {announcement}");
 self.part
 }
}
```
- *Elision rules were applied*

# Mix It with Traits and Generics



- `use std::fmt::Display;`

```
fn longest_with_an_announcement<'a, T>(
 x: &'a str,
 y: &'a str,
 ann: T,
) -> &'a str
where
 T: Display,
{
 println!("Announcement! {ann}");
 if x.len() > y.len() { x } else { y }
}
```

# Static Lifetime



- The reference can live for the entire duration of the program.
- E. g.
  - **let s: &'static str = "I have a static lifetime.";**

# Example Project 1



- **use std::env;  
use std::fs;**

```
fn main() {
 let args: Vec<String> = env::args().collect();
 // dbg!(args);

 let query = &args[1];
 let file_path = &args[2];

 println!("Searching for {query}");
 println!("In file {file_path}");

 let contents = fs::read_to_string(file_path)
 .expect("Should have been able to read the file");

 println!("With text:\n{contents}");
}
```

# Usage



- cargo build
- cargo run -- phrase filename
  - filename in the rust project root folder
- cargo test
  - later
- If debug directive used cannot compile for release

# Example Project 2



- **use std::env;**  
**use std::fs;**

```
fn main() {
 let args: Vec<String> = env::args().collect();

 let config = Config::new(&args);

 println!("Searching for {}", config.query);
 println!("In file {}", config.file_path);

 let contents = fs::read_to_string(config.file_path)
 .expect("Should have been able to read the file");

 println!("With text:\n{contents}");
}
```

# Example Project 2 continued



- ```
struct Config {  
    query: String,  
    file_path: String,  
}  
  
impl Config {  
    fn new(args: &[String]) -> Config {  
        let query = args[1].clone();  
        let file_path = args[2].clone();  
  
        Config { query, file_path }  
    }  
}
```

Example Project 3



- ```
use std::env;
use std::fs;
use std::process;

fn main() {
 let args: Vec<String> = env::args().collect();

 let config = Config::build(&args).unwrap_or_else(|err| {
 println!("Problem parsing arguments: {err}");
 process::exit(1);
 });

 println!("Searching for {}", config.query);
 println!("In file {}", config.file_path);

 let contents = fs::read_to_string(config.file_path)
 .expect("Should have been able to read the file");

 println!("With text:\n{contents}");
}
```



# Example Project 3 continued



- ```
struct Config {  
    query: String,  
    file_path: String,  
}  
  
impl Config {  
    fn build(args: &[String]) -> Result<Config, &'static str> {  
        if args.len() < 3 {  
            return Err("not enough arguments, expected phrase and file");  
        }  
  
        let query = args[1].clone();  
        let file_path = args[2].clone();  
  
        Ok(Config { query, file_path })  
    }  
}
```

Example Project 4



```
• use std::env;
  use std::fs;
  use std::process;
  use std::error::Error;

  struct Config {
      query: String,
      file_path: String,
  }

  impl Config {
      fn build(args: &[String]) -> Result<Config, &'static str> {
          if args.len() < 3 {
              return Err("not enough arguments, expected phrase and file");
          }

          let query = args[1].clone();
          let file_path = args[2].clone();

          Ok(Config { query, file_path })
      }
  }
```

Example Project 4 continued



```
• fn main() {  
    let args: Vec<String> = env::args().collect();  
  
    let config = Config::build(&args).unwrap_or_else(|err| {  
        println!("Problem parsing arguments: {err}");  
        process::exit(1);  
    });  
  
    println!("Searching for {}", config.query);  
    println!("In file {}", config.file_path);  
  
    if let Err(e) = run(config) {  
        println!("Application error: {e}");  
        process::exit(1);  
    }  
}  
  
fn run(config: Config) -> Result<(), Box<dyn Error>> {  
    let contents = fs::read_to_string(config.file_path)?;  
  
    println!("With text:\n{contents}");  
  
    Ok(())  
}
```

Example Library 5 – main.rs



- ```
use std::env;
use std::process;
use example_lib::Config;

fn main() {
 let args: Vec<String> = env::args().collect();

 let config = Config::build(&args).unwrap_or_else(|err| {
 println!("Problem parsing arguments: {err}");
 process::exit(1);
 });

 println!("Searching for {}", config.query);
 println!("In file {}", config.file_path);

 if let Err(e) = example_lib::run(config) {
 println!("Application error: {e}");
 process::exit(1);
 }
}
```

# Example Library 5 – lib.rs



- ```
use std::fs;
use std::error::Error;
```
- ```
pub struct Config {
 pub query: String,
 pub file_path: String,
}
```
- ```
impl Config {
    pub fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("not enough arguments, expected phrase and file");
        }

        let query = args[1].clone();
        let file_path = args[2].clone();

        Ok(Config { query, file_path })
    }
}
```
- ```
pub fn run(config: Config) -> Result<(), Box<dyn Error>> {
 let contents = fs::read_to_string(config.file_path)?;

 println!("With text:\n{contents}");

 Ok(())
}
```

# Example Library 6 – v1.0 – lib.rs



- ```
use std::fs;
use std::error::Error;
```
- ```
pub struct Config {
 pub query: String,
 pub file_path: String,
}
```
- ```
impl Config {
    pub fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("not enough arguments, expected phrase and file");
        }

        let query = args[1].clone();
        let file_path = args[2].clone();

        Ok(Config { query, file_path })
    }
}
```
- ```
pub fn run(config: Config) -> Result<(), Box<dyn Error>> {
 let contents = fs::read_to_string(config.file_path)?;

 for line in search(&config.query, &contents) {
 println!("{line}");
 }

 Ok(())
}
```

# Example Library 6 – v1.0 – lib.rs – cont.



```
• #[cfg(test)]
mod tests {
 use super::*

 #[test]
 fn one_result() {
 let query = "duct";
 let contents = "\

Rust:
safe, fast, productive.
Pick three.";

 assert_eq!(vec!["safe, fast, productive."], search(query, contents));
 }
}
pub fn search<'a>(query: &'a str, contents: &'a str) -> Vec<&'a str> {
 let mut results = Vec::new();
 for line in contents.lines() {
 if line.contains(query) {
 results.push(line);
 }
 }
 results
}
```

# Example Library 7 – v2.0 – main.rs



```
• use std::env;
 use std::process;
 use example_lib::Config;

fn main() {
 let args: Vec<String> = env::args().collect();

 let config = Config::build(&args).unwrap_or_else(|err| {
 eprintln!("Problem parsing arguments: {err}");
 process::exit(1);
 });

 println!("Searching for {}", config.query);
 println!("In file {}", config.file_path);

 if let Err(e) = example_lib::run(config) {
 eprintln!("Application error: {e}");
 process::exit(1);
 }
}
```



# Example Library 7 – v2.0 – lib.rs



- ```
use std::fs;
use std::error::Error;
use std::env;
```
- ```
pub struct Config {
 pub query: String,
 pub file_path: String,
 pub ignore_case: bool,
}
```
- ```
impl Config {
    pub fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("not enough arguments, expected phrase and file");
        }

        let query = args[1].clone();
        let file_path = args[2].clone();

        let ignore_case = env::var("IGNORE_CASE").is_ok();

        Ok(Config { query, file_path, ignore_case })
    }
}
```

Example Library 7 – v2.0 – lib.rs – cont.



- ```
pub fn run(config: Config) -> Result<(), Box<dyn Error>> {
 let contents = fs::read_to_string(config.file_path)?;

 let results = if config.ignore_case {
 search_case_insensitive(&config.query, &contents)
 } else {
 search(&config.query, &contents)
 };

 for line in results {
 println!("{line}");
 }

 Ok(())
}
```

# Example Library 7 – v2.0 – lib.rs – cont.



- ```
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {  
    let mut results = Vec::new();  
    for line in contents.lines() {  
        if line.contains(query) {  
            results.push(line);  
        }  
    }  
    results  
}
```
- ```
pub fn search_case_insensitive<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
 let query = query.to_lowercase();
 let mut results = Vec::new();

 for line in contents.lines() {
 if line.to_lowercase().contains(&query) {
 results.push(line);
 }
 }

 results
}
```

# Example Library 7 – v2.0 – lib.rs – test 1



- ```
#[cfg(test)]  
mod tests {  
    use super::*;  
  
    #[test]  
    fn one_result() {  
        let query = "duct";  
        let contents = "\n  
Rust:  
safe, fast, productive.  
Pick three."  
  
        assert_eq!(vec!["safe, fast, productive."],  
                   search(query, contents));  
    }  
}
```

Example Library 7 – v2.0 – lib.rs – test 2



- ```
#[test]
fn case_insensitive() {
 let query = "rUsT";
 let contents = "\

Rust:
safe, fast, productive.
Pick three.
Trust me.";

 assert_eq!(
 vec!["Rust:", "Trust me."],
 search_case_insensitive(query, contents)
);

 }

}
```

# Usage of v2.0



- `cargo run -- to poem.txt`
- `IGNORE_CASE=1 cargo run -- to poem.txt`
- `cargo test`

# Functional Features



- Closures
  - common feature in FP, a programmer even doesn't notice
- *Rust's closures are anonymous functions you can save in a variable or pass as arguments to other functions.*
- Iterators
  - in FP, map, fold\*, etc.
- *The iterator pattern allows you to perform some task on a sequence of items in turn. ... In Rust, iterators are lazy, meaning they have no effect until you call methods that consume the iterator to use it up.*

# Closures – Example



- **`#[derive(Debug, PartialEq, Copy, Clone)]`**  
**`enum ShirtColor {`**  
    **`Red,`**  
    **`Blue,`**  
**`}`**  
  
**`struct Inventory {`**  
    **`shirts: Vec<ShirtColor>,`**  
**`}`**



# Closures – Example (continued)



- ```
impl Inventory {  
    fn giveaway(&self, user_preference: Option<ShirtColor>) -> ShirtColor {  
        user_preference.unwrap_or_else(|| self.most_stocked())  
    }  
  
    fn most_stocked(&self) -> ShirtColor {  
        let mut num_red = 0;  
        let mut num_blue = 0;  
  
        for color in &self.shirts {  
            match color {  
                ShirtColor::Red => num_red += 1,  
                ShirtColor::Blue => num_blue += 1,  
            }  
        }  
        if num_red > num_blue {  
            ShirtColor::Red  
        } else {  
            ShirtColor::Blue  
        }  
    }  
}
```

Closures – Example (continued)



- ```
fn main() {
 let store = Inventory {
 shirts: vec![ShirtColor::Blue, ShirtColor::Red, ShirtColor::Blue],
 };

 let user_pref1 = Some(ShirtColor::Red);
 let giveaway1 = store.giveaway(user_pref1);
 println!(
 "The user with preference {:?} gets {:?}",
 user_pref1, giveaway1
);

 let user_pref2 = None;
 let giveaway2 = store.giveaway(user_pref2);
 println!(
 "The user with preference {:?} gets {:?}",
 user_pref2, giveaway2
);
}
```

# Closures and Functions



- No need of type annotation

- Fully typed

- ```
let expensive_closure = |num: u32| -> u32 {  
    println!("calculating slowly...");  
    thread::sleep(Duration::from_secs(2));  
    num  
};
```

- Simplification

- ```
fn add_one_v1 (x: u32) -> u32 { x + 1 }
```
    - ```
let add_one_v2 = |x: u32| -> u32 { x + 1 };
```
 - ```
let add_one_v3 = |x| { x + 1 };
```
    - ```
let add_one_v4 = |x|           x + 1 ;
```

Danger!!



- Rust is not Haskell!
 - Type inference and instantiation just once

- **let example_closure = |x| x;**

```
let s = example_closure(String::from("hello"));  
let n = example_closure(5);
```



Borrowing



- This will work:
- ```
fn main() {
 let list = vec![1, 2, 3];
 println!("Before defining closure: {list:?}");

 let only_borrows = || println!("From closure: {list:?}");

 println!("Before calling closure: {list:?}");
 only_borrows();
 println!("After calling closure: {list:?}");
}
```

## Borrowing 2



- This will also work:
- ```
fn main() {  
    let mut list = vec![1, 2, 3];  
    println!("Before defining closure: {list:?}");  
  
    let mut borrows_mutably = || list.push(7);  
    // no println! , wouldn't work  
    borrows_mutably();  
    println!("After calling closure: {list:?}");  
}
```

Take Ownership



- Use keyword **move**, e.g.:
- **use std::thread;**

```
fn main() {  
    let list = vec![1, 2, 3];  
    println!("Before defining closure: {list:?}");  
  
    thread::spawn(move || println!("From thread: {list:?}"))  
        .join()  
        .unwrap();  
}
```

Take Ownership – Run



- `$ cargo build`
Compiling move_owner v0.1.0
(/home/kolar/Vyuka/FLP/Rust/move_owner)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.55s
- `$ cargo run`
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
Running `target/debug/move\_owner`
Before defining closure: [1, 2, 3]
From thread: [1, 2, 3]

Moving Things Out



- Fn Traits
- **FnOnce** applies to closures that can be called once. All closures implement at least this trait because all closures can be called. A closure that moves captured values out of its body will only implement FnOnce and none of the other Fn traits, because it can only be called once.
- **FnMut** applies to closures that don't move captured values out of their body, but that might mutate the captured values. These closures can be called more than once.
- **Fn** applies to closures that don't move captured values out of their body and that don't mutate captured values, as well as closures that capture nothing from their environment. These closures can be called more than once without mutating their environment, which is important in cases such as calling a closure multiple times concurrently.

FnOnce in Option<T>



- ```
impl<T> Option<T> {
 pub fn unwrap_or_else<F>(self, f: F) -> T
 where
 F: FnOnce() -> T
 {
 match self {
 Some(x) => x,
 None => f(),
 }
 }
}
```

# FnMut in Sort



- ```
#[derive(Debug)]  
struct Rectangle {  
    width: u32,  
    height: u32,  
}  
  
fn main() {  
    let mut list = [  
        Rectangle { width: 10, height: 1 },  
        Rectangle { width: 3, height: 5 },  
        Rectangle { width: 7, height: 12 },  
    ];  
  
    list.sort_by_key(|r| r.width);  
    println!("{list:#?}");  
}
```

FnMut Example Output



- ```
$ cargo run
 Compiling rectangles v0.1.0 (file:///projects/rectangles)
 Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.41s
 Running `target/debug/rectangles`
[
 Rectangle {
 Width: 3,
 Height: 5,
 },
 Rectangle {
 Width: 7,
 Height: 12,
 },
 Rectangle {
 Width: 10,
 Height: 1,
 },
]
```

# FnOnce is Over FnMut



- ```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    let mut sort_operations = vec![];
    let value = String::from("closure called");

    list.sort_by_key(|r| {
        sort_operations.push(value);
        r.width
    });
    println!("{list:#?}");
}
```



OK, Still FnMut



- ```
#[derive(Debug)]
struct Rectangle {
 width: u32,
 height: u32,
}

fn main() {
 let mut list = [
 Rectangle { width: 10, height: 1 },
 Rectangle { width: 3, height: 5 },
 Rectangle { width: 7, height: 12 },
];

 let mut num_sort_operations = 0;
 list.sort_by_key(|r| {
 num_sort_operations += 1;
 r.width // even r.width*r.height
 });
 println!("{list:#?}, sorted in {num_sort_operations} operations");
}
```

# Iterators



- ```
fn main() {  
    let v1 = vec![1, 2, 3];  
  
    let v1_iter = v1.iter();    // creating an iterator  
  
    for val in v1_iter {        // using an iterator  
        println!("Got: {val}");  
    }  
}
```

Iterator Trait



- ```
pub trait Iterator {
 type Item;

 fn next(&mut self) -> Option<Self::Item>;

 // methods with default implementations elided
}
```
- **type Item** and **Self::Item** are defining an associated type with this trait



# Iterator Test



- **`#[test]`**  
**`fn iterator_demonstration() {`**  
    **`let v1 = vec![1, 2, 3];`**  
  
    **`let mut v1_iter = v1.iter();`**  
  
    **`assert_eq!(v1_iter.next(), Some(&1));`**  
    **`assert_eq!(v1_iter.next(), Some(&2));`**  
    **`assert_eq!(v1_iter.next(), Some(&3));`**  
    **`assert_eq!(v1_iter.next(), None);`**  
**`}`**

# Iterator Consumers – sum



- ```
#[test]  
fn iterator_sum() {  
    let v1 = vec![1, 2, 3];  
  
    let v1_iter = v1.iter();  
  
    let total: i32 = v1_iter.sum(); // consumes iterator  
  
    assert_eq!(total, 6);  
}
```

Iterators Re-Production



- ```
let v1: Vec<i32> = vec![1, 2, 3];

let v2: Vec<_> = v1.iter().map(|x| x + 1).collect();

assert_eq!(v2, vec![2, 3, 4]);
```
- 
- Without red text nothing happens, iterators are lazy.

# Closures in Iterators



- ```
#[derive(PartialEq, Debug)]  
struct Shoe {  
    size: u32,  
    style: String,  
}  
  
fn shoes_in_size(shoes: Vec<Shoe>, shoe_size: u32) -> Vec<Shoe> {  
    shoes.into_iter().filter(|s| s.size == shoe_size).collect()  
}
```

Testing...



```
• #[cfg(test)]
  mod tests {
      use super::*;

      #[test]
      fn filters_by_size() {
          let shoes = vec![
              Shoe {
                  size: 10, style: String::from("sneaker"),
              },
              Shoe {
                  size: 13, style: String::from("sandal"),
              },
              Shoe {
                  size: 10, style: String::from("boot"),
              },
          ];

          let in_my_size = shoes_in_size(shoes, 10);

          assert_eq!(
              in_my_size,
              vec![
                  Shoe {
                      size: 10, style: String::from("sneaker")
                  },
                  Shoe {
                      size: 10, style: String::from("boot")
                  },
              ]
          );
      }
  }
```

Example – Re-Iterated over Closures



- ```
impl Config {
 pub fn build(
 mut args: impl Iterator<Item = String>,
) -> Result<Config, &'static str> {
 args.next();

 let query = match args.next() {
 Some(arg) => arg,
 None => return Err("Didn't get a query string"),
 };

 let file_path = match args.next() {
 Some(arg) => arg,
 None => return Err("Didn't get a file path"),
 };

 let ignore_case = env::var("IGNORE_CASE").is_ok();

 Ok(Config { query, file_path, ignore_case })
 }
}
```

# Example – Re-Iterated over Closures 2



- ```
fn main() {  
    let config = Config::build(env::args()).unwrap_or_else(|err| {  
        eprintln!("Problem parsing arguments: {err}");  
        process::exit(1);  
    });  
  
    println!("Searching for {}", config.query);  
    println!("In file {}", config.file_path);  
  
    if let Err(e) = example_iter::run(config) {  
        eprintln!("Application error: {e}");  
        process::exit(1);  
    }  
}
```

Example – Re-Iterated over Closures 3



- ```
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
 contents
 .lines()
 .filter(|line| line.contains(query))
 .collect()
}
```

```
pub fn search_case_insensitive<'a>(
 query: &str,
 contents: &'a str,
) -> Vec<&'a str> {
 let query = query.to_lowercase();

 contents
 .lines()
 .filter(|line| line.to_lowercase().contains(&query))
 .collect()
}
```



# Smart Pointers`



- You've heard about it, probably (C++)
- Rust:
  - `Box<T>`, *for allocating values on the heap*
  - `Rc<T>`, *a reference counting type that enables multiple ownership*
  - `Ref<T>` and `RefMut<T>`, *accessed through `RefCell<T>`, a type that enforces the borrowing rules at runtime instead of compile time*

# Box<T> – Reference on Heap



- ```
fn main() {  
    let b = Box::new(5);  
    println!("b = {b}");  
}
```
- Recursive data types
 - Not as simple as in Haskell
 - Imperative language, thus something like pointers must be used

Box<T> – Recursive Data Types



- **enum List {
 Cons(i32, Box<List>),
 Nil,
}**
- **use crate::List::{Cons, Nil};**

```
fn main() {  
    let list = Cons(1, Box::new(  
        Cons(2, Box::new(  
            Cons(3, Box::new(Nil)))))  
);  
}
```

Following Pointers – Box<T>



- ```
fn main() {
 let x = 5;
 let y = &x;
 assert_eq!(5, x);
 assert_eq!(5, *y);
}
```
- ```
fn main() {  
    let x = 5;  
    let y = Box::new(x);  
    assert_eq!(5, x);  
    assert_eq!(5, *y);  
}
```

My Own Box



- **struct MyBox<T>(T);**
- **impl<T> MyBox<T> {
 fn new(x: T) -> MyBox<T> {
 MyBox(x)
 }
}**
- **use std::ops::Deref;**

**impl<T> Deref for MyBox<T> {
 type Target = T;

 fn deref(&self) -> &Self::Target {
 &self.0
 }
}**

Rust Coercion



- From `&T` to `&U` when `T: Deref<Target=U>`
- From `&mut T` to `&mut U` when `T: DerefMut<Target=U>`
- From `&mut T` to `&U` when `T: Deref<Target=U>`

Drop Trait



```
• struct CustomSmartPointer {  
    data: String,  
}  
  
impl Drop for CustomSmartPointer {  
    fn drop(&mut self) {  
        println!("Dropping CustomSmartPointer with data `{}`!",  
                self.data);  
    }  
}  
  
fn main() {  
    let c = CustomSmartPointer {  
        data: String::from("my stuff"),  
    };  
    let d = CustomSmartPointer {  
        data: String::from("other stuff"),  
    };  
    println!("CustomSmartPointers created.");  
}
```

Drop Trait Run



- `$ cargo run`
Compiling drop-example v0.1.0 (file:///projects/drop-example)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.60s
Running `target/debug/drop-example`
CustomSmartPointers created.
Dropping CustomSmartPointer with data `other stuff`!
Dropping CustomSmartPointer with data `my stuff`!

Explicit Drop



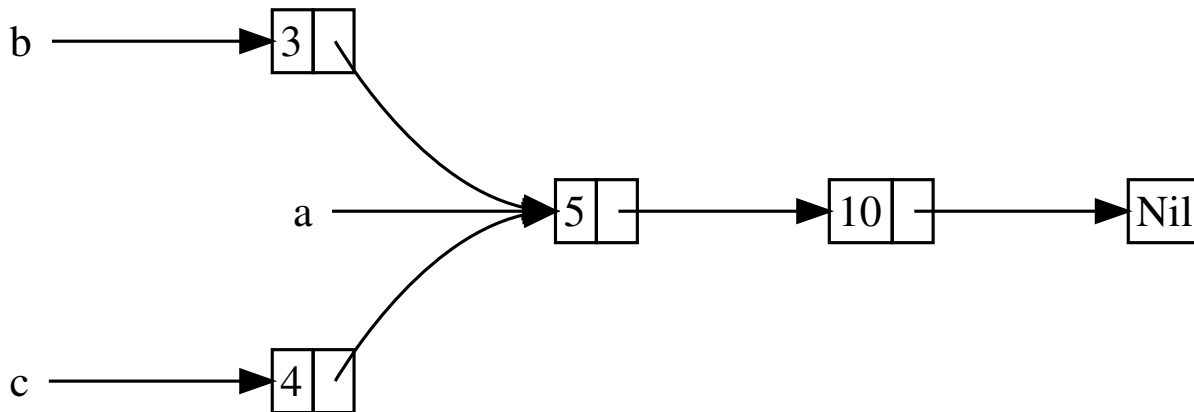
- ```
fn main() {
 let c = CustomSmartPointer {
 data: String::from("some data"),
 };
 println!("CustomSmartPointer created.");
 drop(c);
 println!("CustomSmartPointer dropped before the end of main.");
}
```

# Explicit Drop Run



- `$ cargo run`  
Compiling drop-example v0.1.0 (file:///projects/drop-example)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.73s  
Running `target/debug/drop-example`  
CustomSmartPointer created.  
Dropping CustomSmartPointer with data `some data`!  
CustomSmartPointer dropped before the end of main.

# Rc<T> – Reference Counted Smart Pointer



This cannot be done directly (e.g. as in Haskell).  
Box<T> is too weak for this.

# Rc<T> – List



- ```
enum List {  
    Cons(i32, Rc<List>),  
    Nil,  
}
```
- ```
use crate::List::{Cons, Nil};
use std::rc::Rc;

fn main() {
 let a = Rc::new(Cons(5, Rc::new(Cons(10, Rc::new(Nil)))))
 let b = Cons(3, Rc::clone(&a));
 let c = Cons(4, Rc::clone(&a));
}
```

# Rc<T> – Counting the References



- ```
fn main() {  
    let a = Rc::new(Cons(5, Rc::new(Cons(10, Rc::new(Nil)))));  
    println!("count after creating a = {}", Rc::strong_count(&a));  
  
    let b = Cons(3, Rc::clone(&a));  
    println!("count after creating b = {}", Rc::strong_count(&a));  
  
    {  
        let c = Cons(4, Rc::clone(&a));  
        println!("count after creating c = {}", Rc::strong_count(&a));  
    }  
  
    println!("count after c goes out of scope = {}", Rc::strong_count(&a));  
}
```

Rc<T> – Run the Counter



- `$ cargo run`
Compiling cons-list v0.1.0 (file:///projects/cons-list)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.45s
Running `target/debug/cons-list`
count after creating a = 1
count after creating b = 2
count after creating c = 3
count after c goes out of scope = 2

RefCell<T> – Runtime Borrowing



- Reminder:
 - *At any given time, you can have either one mutable reference or any number of immutable references (but not both).*
 - *References must always be valid.*
- With **references** and **Box<T>**, the borrowing rules' invariants are enforced at **compile time**. With **RefCell<T>**, these invariants are enforced at **runtime**.
- With **references**, if you break these rules, you'll get a **compiler error**. With **RefCell<T>**, if you break these rules, your **program will panic and exit**.

RefCell<T> – Selector Guide



- **Rc<T>** enables **multiple owners of the same data**; **Box<T>** and **RefCell<T>** have **single owners**.
- **Box<T>** allows **immutable or mutable borrows** checked at compile time; **Rc<T>** allows **only immutable borrows** checked at compile time; **RefCell<T>** allows **immutable or mutable borrows** checked at runtime.
- Because **RefCell<T>** allows mutable borrows checked at runtime, you can mutate the value inside the **RefCell<T>** even when the **RefCell<T>** is immutable.

RefCell<T> – Example



- ```
pub trait Messenger {
 fn send(&self, msg: &str);
}

pub struct LimitTracker<'a, T: Messenger> {
 messenger: &'a T,
 value: usize,
 max: usize,
}
```

# RefCell<T> – Example (continued)



- ```
impl<'a, T> LimitTracker<'a, T>
where
    T: Messenger,
{
    pub fn new(messenger: &'a T, max: usize)
        -> LimitTracker<'a, T>
    {
        LimitTracker {
            messenger,
            value: 0,
            max,
        }
    }
}
```
- ...

RefCell<T> – Example (continued)



- ```
pub fn set_value(&mut self, value: usize) {
 self.value = value;

 let percentage_of_max = self.value as f64 / self.max as f64;

 if percentage_of_max >= 1.0 {
 self.messenger.send("Error: You are over your quota!");
 } else if percentage_of_max >= 0.9 {
 self.messenger
 .send("Urgent: You've used up over 90% of your quota!");
 } else if percentage_of_max >= 0.75 {
 self.messenger
 .send("Warning: You've used up over 75% of your quota!");
 }
}

} // of impl
```

# RefCell<T> – Test



- ```
#[cfg(test)]
mod tests {
    use super::*;
    use std::cell::RefCell;

    struct MockMessenger {
        sent_messages: RefCell<Vec<String>>,
    }

    impl MockMessenger {
        fn new() -> MockMessenger {
            MockMessenger {
                sent_messages: RefCell::new(vec![]),
            }
        }
    }
}
```

RefCell<T> – Test (continued)



- ```
impl Messenger for MockMessenger {
 fn send(&self, message: &str) {
 self.sent_messages.borrow_mut().push(String::from(message));
 }
}

#[test]
fn it_sends_an_over_75_percent_warning_message() {
 let mock_messenger = MockMessenger::new();
 let mut limit_tracker = LimitTracker::new(&mock_messenger, 100);

 limit_tracker.set_value(80);

 assert_eq!(mock_messenger.sent_messages.borrow().len(), 1);
}
} // mod tests
```

# Allowing Multiple Owners of Mutable Data with Rc<T> and RefCell<T>



- ```
#[derive(Debug)]
enum List {
    Cons(Rc<RefCell<i32>>, Rc<List>),
    Nil,
}
```
- ```
use crate::List::{Cons, Nil};
use std::cell::RefCell;
use std::rc::Rc;
```
- ```
fn main() {
    let value = Rc::new(RefCell::new(5));
    let a = Rc::new(Cons(Rc::clone(&value), Rc::new(Nil)));
    let b = Cons(Rc::new(RefCell::new(3)), Rc::clone(&a));
    let c = Cons(Rc::new(RefCell::new(4)), Rc::clone(&a));

    *value.borrow_mut() += 10;

    println!("a after = {a:?}");
    println!("b after = {b:?}");
    println!("c after = {c:?}");
}
```

Running Previous Example



- `$ cargo run`
 Compiling cons-list v0.1.0 (file:///projects/cons-list)
 Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.63s
 Running `target/debug/cons-list`
a after = Cons(RefCell { value: 15 }, Nil)
b after = Cons(RefCell { value: 3 }, Cons(RefCell { value: 15 }, Nil))
c after = Cons(RefCell { value: 4 }, Cons(RefCell { value: 15 }, Nil))

Reference Cycles Can Leak Memory



- ```
use crate::List::{Cons, Nil};
use std::cell::RefCell;
use std::rc::Rc;

#[derive(Debug)]
enum List {
 Cons(i32, RefCell<Rc<List>>), Nil,
}

impl List {
 fn tail(&self) -> Option<&RefCell<Rc<List>>> { // Haskell tail
 match self {
 Cons(_, item) => Some(item),
 Nil => None,
 }
 }
}
```



# Creating a Cycle



```
• fn main() {
 let a = Rc::new(Cons(5, RefCell::new(Rc::new(Nil))));

 println!("a initial rc count = {}", Rc::strong_count(&a));
 println!("a next item = {:?}", a.tail());

 let b = Rc::new(Cons(10, RefCell::new(Rc::clone(&a))));

 println!("a rc count after b creation = {}", Rc::strong_count(&a));
 println!("b initial rc count = {}", Rc::strong_count(&b));
 println!("b next item = {:?}", b.tail());

 if let Some(link) = a.tail() {
 *link.borrow_mut() = Rc::clone(&b);
 }

 println!("b rc count after changing a = {}", Rc::strong_count(&b));
 println!("a rc count after changing a = {}", Rc::strong_count(&a));

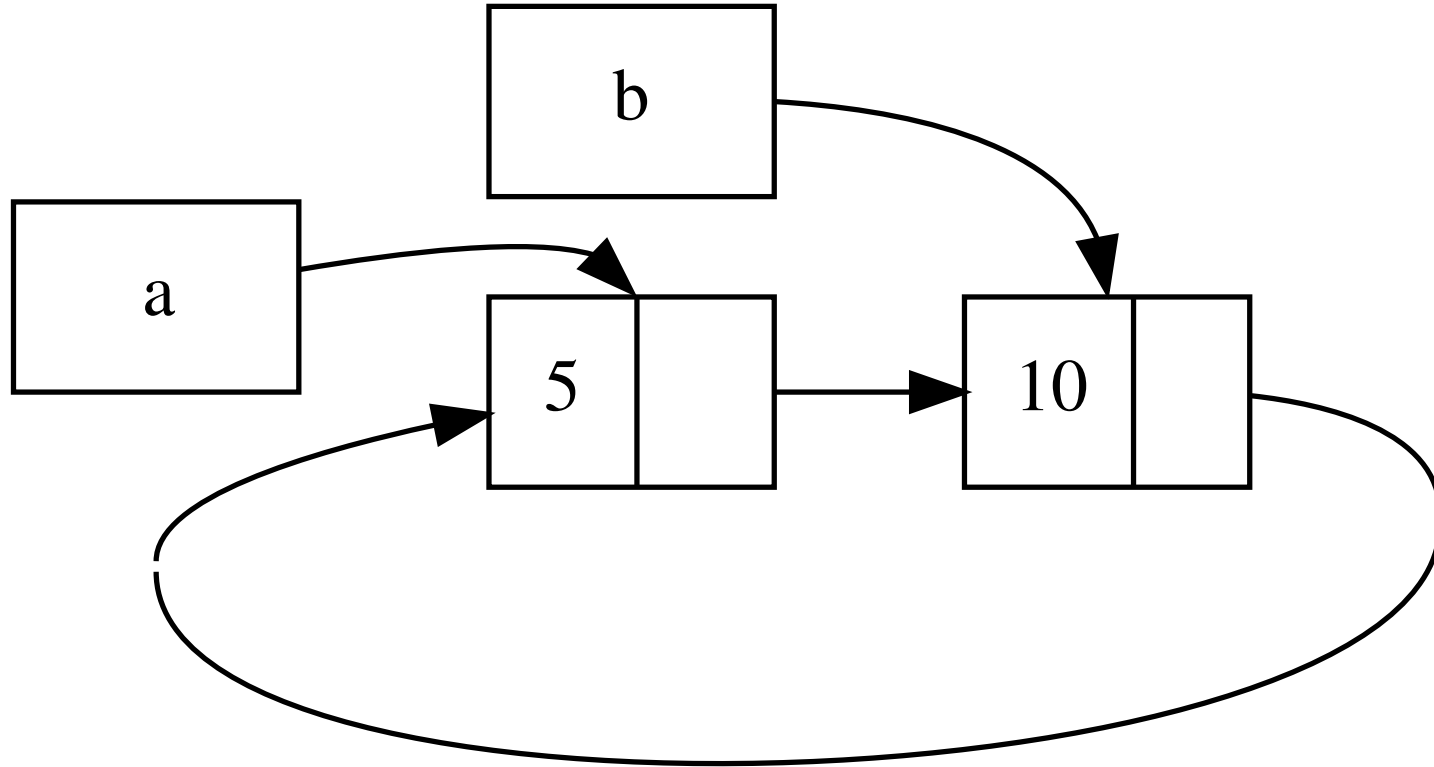
 // Uncomment the next line to see that we have a cycle;
 // it will overflow the stack.
 // println!("a next item = {:?}", a.tail());
}
```

# Running Previous Example



- `$ cargo run`  
Compiling cons-list v0.1.0 (file:///projects/cons-list)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.53s  
Running `target/debug/cons-list`  
a initial rc count = 1  
a next item = Some(RefCell { value: Nil })  
a rc count after b creation = 2  
b initial rc count = 1  
b next item = Some(RefCell { value: Cons(5, RefCell { value: Nil }) })  
b rc count after changing a = 2  
a rc count after changing a = 2

# Cycle



# Cycles in Memory



- Sometimes it is not desired...
- Sometimes it is desired.
  - Can we make it another way?
  - Weak<T> can handle cycles and memory
    - See doc.

# Concurrency



- Power of Rust
  - How to create threads to run multiple pieces of code at the same time
  - Message-passing concurrency, where channels send messages between threads
  - Shared-state concurrency, where multiple threads have access to some piece of data
  - The Sync and Send traits, which extend Rust's concurrency guarantees to user-defined types as well as types provided by the standard library

# Creating a New Thread



- **use std::thread;**  
**use std::time::Duration;**

```
fn main() {
 thread::spawn(|| {
 for i in 1..10 {
 println!("hi number {i} from the spawned thread!");
 thread::sleep(Duration::from_millis(1));
 }
 });

 for i in 1..5 {
 println!("hi number {i} from the main thread!");
 thread::sleep(Duration::from_millis(1));
 }
}
```

# Waiting for All Threads



- **use std::thread;**  
**use std::time::Duration;**

```
fn main() {
 let handle = thread::spawn(|| {
 for i in 1..10 {
 println!("hi number {i} from the spawned thread!");
 thread::sleep(Duration::from_millis(1));
 }
 });

 for i in 1..5 {
 println!("hi number {i} from the main thread!");
 thread::sleep(Duration::from_millis(1));
 }

 handle.join().unwrap();
}
```

# Waiting Elsewhere



- **use std::thread;**  
**use std::time::Duration;**

```
fn main() {
 let handle = thread::spawn(|| {
 for i in 1..10 {
 println!("hi number {i} from the spawned thread!");
 thread::sleep(Duration::from_millis(1));
 }
 });

 handle.join().unwrap();

 for i in 1..5 {
 println!("hi number {i} from the main thread!");
 thread::sleep(Duration::from_millis(1));
 }
}
```



# Move Closures



- Straightforward and simplest way is the best
- **use `std::thread`;**

```
fn main() {
 let v = vec![1, 2, 3];

 let handle = thread::spawn(move || {
 println!("Here's a vector: {v:?}");
 });

 handle.join().unwrap();
}
```

# Message Passing



- Useless example 😊
- **use std::sync::mpsc;**  
**use std::thread;**

```
fn main() {
 let (tx, rx) = mpsc::channel();

 thread::spawn(move || {
 let val = String::from("hi");
 tx.send(val).unwrap(); // Result<T, E>
 });
}
```

# Receiving the Message



- **use std::sync::mpsc;**  
**use std::thread;**

```
fn main() {
 let (tx, rx) = mpsc::channel();

 thread::spawn(move || {
 let val = String::from("hi");
 tx.send(val).unwrap();
 });

 // try_recv - non-blocking
 let received = rx.recv().unwrap(); // Result<T,E>
 println!("Got: {received}");
}
```

# Sending Several Messages, Pausing



- ```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let vals = vec![
            String::from("hi"), String::from("from"),
            String::from("the"), String::from("thread"),
        ];

        for val in vals {
            tx.send(val).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });

    for received in rx {
        println!("Got: {received}");
    }
}
```

Several Transmitters



- ...
let (tx, rx) = mpsc::channel();
let tx1 = tx.clone();
...

Mutexes



- Simple one (useless 😊)
- **use std::sync::Mutex;**

```
fn main() {  
    let m = Mutex::new(5);  
  
    {  
        let mut num = m.lock().unwrap();  
        *num = 6;  
    }  
  
    println!("m = {m:?}");  
}
```

Mutexes – Several Threads – Arc<T>



- ```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
 let counter = Arc::new(Mutex::new(0));
 let mut handles = vec![];

 for _ in 0..10 {
 let counter = Arc::clone(&counter);
 let handle = thread::spawn(move || {
 let mut num = counter.lock().unwrap();

 *num += 1;
 });
 handles.push(handle);
 }

 for handle in handles {
 handle.join().unwrap();
 }

 println!("Result: {}", *counter.lock().unwrap());
}
```

# My Own Concurrency



- Using traits `Send` and `Sync` from `std::marker`
  - The `Send` marker trait indicates that ownership of values of the type implementing `Send` can be transferred between threads.
    - (Almost every Rust type is `Send`)
  - The `Sync` marker trait indicates that it is safe for the type implementing `Sync` to be referenced from multiple threads.
- *Because types composed entirely of other types that implement the `Send` and `Sync` traits also automatically implement `Send` and `Sync`, we don't have to implement those traits manually. As marker traits, they don't even have any methods to implement. They're just useful for enforcing invariants related to concurrency. Manually implementing these traits involves implementing unsafe Rust code. The important information is that building new concurrent types not made up of `Send` and `Sync` parts requires careful thought to uphold the safety guarantees. “**The Rustonomicon**” has more information about these guarantees and how to uphold them.*

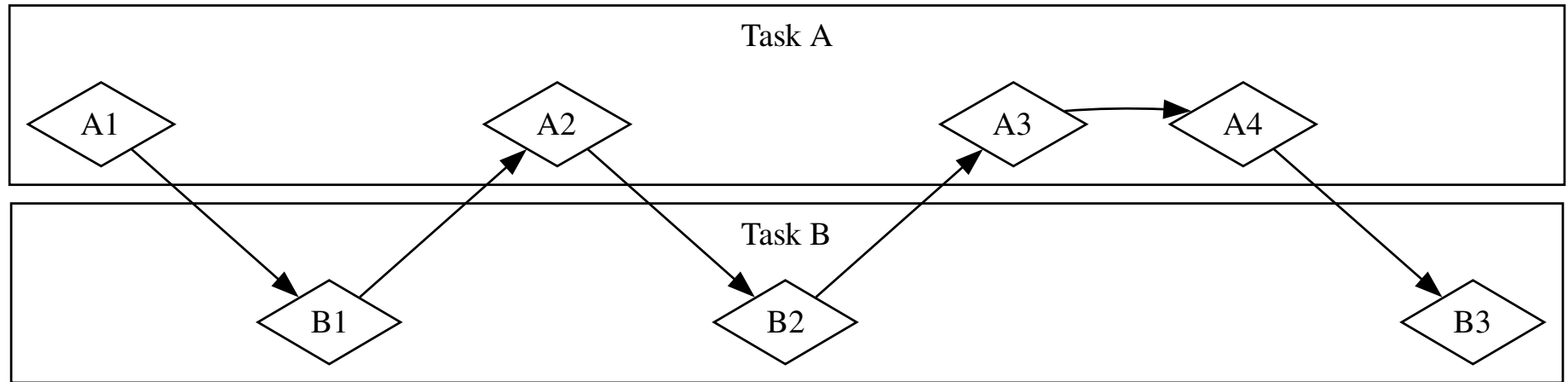


# Asynchronous Operations

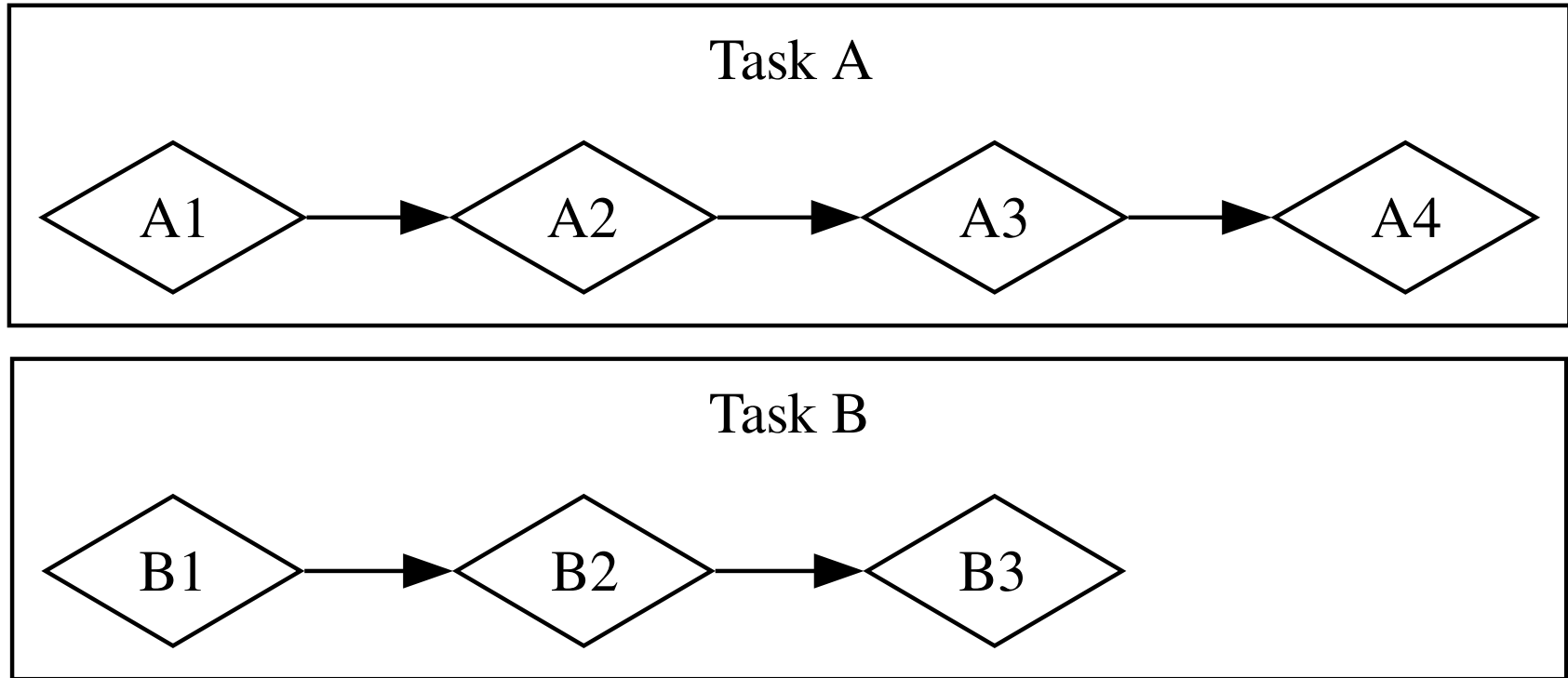


- Today: many cores (even CPUs), tasks running concurrently, in parallel.
- If there is something to do and we have still many things to do why should we wait?
  - E. g. data parallel Haskell
- How to handle this?
  - Too many threads may not do the work faster
  - Low number of threads means longer wait stop computation

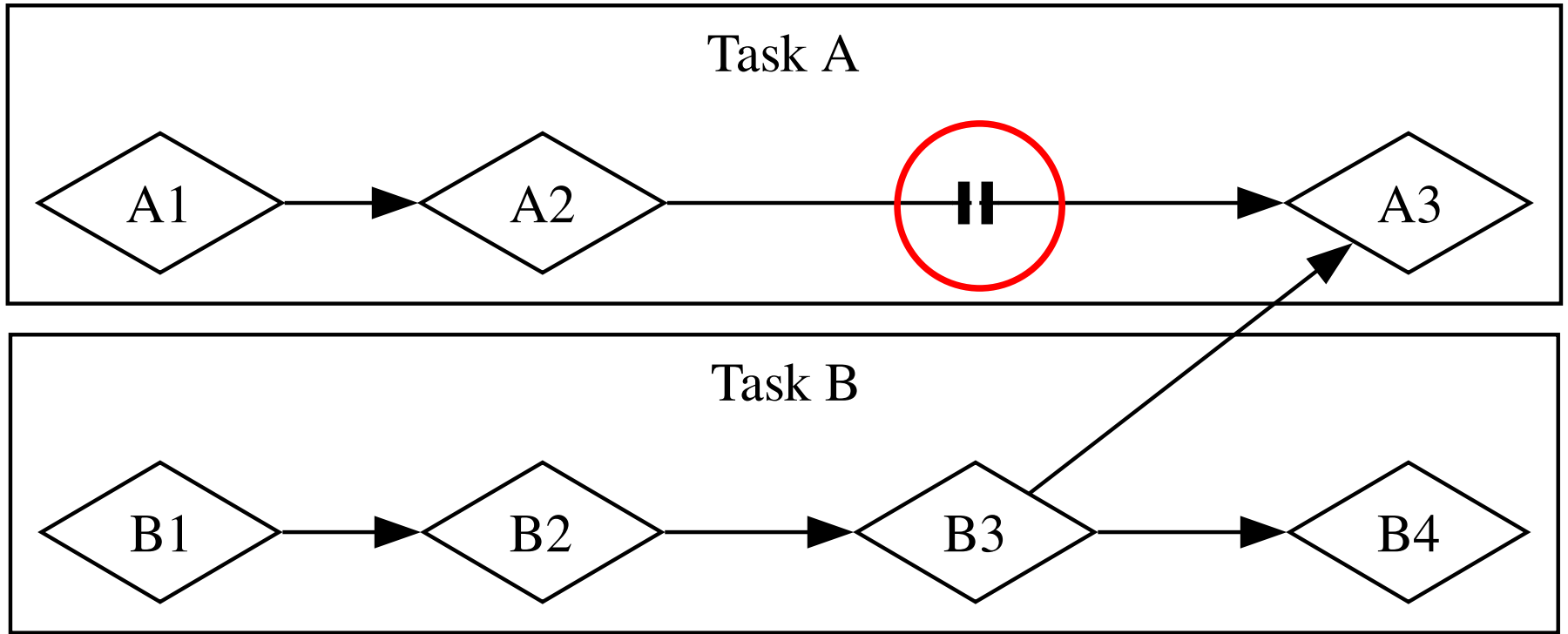
# Concurrent



# Parallel



# Parallel, Waiting



# Futures, Async



- A future is a value that may not be ready now but will become ready at some point in the future.
  - Rust provides a Future trait
- *You can apply the **async** keyword to blocks and functions to specify that they can be interrupted and resumed. Within an async block or async function, you can use the **await** keyword to await a future (that is, wait for it to become ready).*
- **Note:** Some other languages, such as **C#** and **JavaScript**, also use **async** and **await** keywords for async programming. **If you're familiar with those languages, you may notice some significant differences in how Rust does things**, including how it handles the syntax. That's for good reason, as we'll see!

# trpl



- The Rust Programming Language crate – trpl
- `cargo add trpl`

# The Very First Async



- `use trpl::Html;`

```
async fn page_title(url: &str) -> Option<String> {
 let response = trpl::get(url).await;
 // can be chained
 let response_text = response.text().await;
 Html::parse(&response_text)
 .select_first("title")
 .map(|title_element| title_element.inner_html())
}
```

# The Very First Async (continued)



- ```
fn main() {  
    let args: Vec<String> = std::env::args().collect();  
  
    trpl::run(async { // one cannot run await outside async  
        let url = &args[1];  
        match page_title(url).await {  
            Some(title) => println!("The title for {url} was {title}"),  
            None => println!("{url} had no title"),  
        }  
    })  
}
```


Races



- `use trpl::{Either, Html};`

```
async fn page_title(url: &str) -> (&str, Option<String>) {  
    let text = trpl::get(url).await.text().await;  
    let title = Html::parse(&text)  
        .select_first("title")  
        .map(|title| title.inner_html());  
    (url, title)  
}
```

Races (continued)



```
• fn main() {  
    let args: Vec<String> = std::env::args().collect();  
  
    trpl::run(async {  
        let title_fut_1 = page_title(&args[1]);  
        let title_fut_2 = page_title(&args[2]);  
  
        let (url, maybe_title) =  
            match trpl::race(title_fut_1, title_fut_2).await {  
                Either::Left(left) => left,  
                Either::Right(right) => right,  
            };  
  
        println!("{url} returned first");  
        match maybe_title {  
            Some(title) => println!("Its page title is: '{title}'"),  
            None => println!("Its title could not be parsed."),  
        }  
    })  
}
```

spawn_task



- `use std::time::Duration;`

```
fn main() {  
    trpl::run(async {  
        trpl::spawn_task(async {  
            for i in 1..10 {  
                println!("hi number {i} from the first task!");  
                trpl::sleep(Duration::from_millis(500)).await;  
            }  
        });  
  
        for i in 1..5 {  
            println!("hi number {i} from the second task!");  
            trpl::sleep(Duration::from_millis(500)).await;  
        }  
    });  
}
```

spawn_task and await



- use `std::time::Duration`;

```
fn main() {  
    let handle = trpl::spawn_task(async {  
        for i in 1..10 {  
            println!("hi number {i} from the first task!");  
            trpl::sleep(Duration::from_millis(500)).await;  
        }  
    });  
  
    for i in 1..5 {  
        println!("hi number {i} from the second task!");  
        trpl::sleep(Duration::from_millis(500)).await;  
    }  
  
    handle.await.unwrap();  
}
```

Joining 2 Spawns to Single One



- ```
let fut1 = async {
 for i in 1..10 {
 println!("hi number {i} from the first task!");
 trpl::sleep(Duration::from_millis(500)).await;
 }
};

let fut2 = async {
 for i in 1..5 {
 println!("hi number {i} from the second task!");
 trpl::sleep(Duration::from_millis(500)).await;
 }
};

trpl::join(fut1, fut2).await; // result thrown away
```

# Homework



- Remove the `async` block from around either or both of the loops.
- `Await` each `async` block immediately after defining it.
- Wrap only the first loop in an `async` block, and `await` the resulting future after the body of second loop.

# Message Passing with trpl



- Basic, simple:

```
• let (tx, mut rx) = trpl::channel();

 let vals = vec![
 String::from("hi"),
 String::from("from"),
 String::from("the"),
 String::from("future"),
];

 for val in vals {
 tx.send(val).unwrap();
 trpl::sleep(Duration::from_millis(500)).await;
 }

 while let Some(value) = rx.recv().await {
 println!("received '{value}'");
 }
```

# Message Passing with async Blocks



- ```
let (tx, mut rx) = trpl::channel();

let vals = vec![
    String::from("hi"),
    String::from("from"),
    String::from("the"),
    String::from("future"),
];

for val in vals {
    tx.send(val).unwrap();
    trpl::sleep(Duration::from_millis(500)).await;
}

while let Some(value) = rx.recv().await {
    println!("received '{value}'");
}
```

Never stops!

Message Passing with async Blocks



```
let tx_fut = async {  
  let vals = vec![  
    String::from("hi"),  
    String::from("from"),  
    String::from("the"),  
    String::from("future"),  
  ];  
  
  for val in vals {  
    tx.send(val).unwrap();  
    trpl::sleep(Duration::from_millis(500)).await;  
  }  
};  
  
let rx_fut = async {  
  while let Some(value) = rx.recv().await {  
    println!("received '{value}'");  
  }  
};  
  
trpl::join(tx_fut, rx_fut).await;
```

Also never
stops!

Message Passing with async Blocks



```
let tx_fut = async move {  
  let vals = vec![  
    String::from("hi"),  
    String::from("from"),  
    String::from("the"),  
    String::from("future"),  
  ];  
  
  for val in vals {  
    tx.send(val).unwrap();  
    trpl::sleep(Duration::from_millis(500)).await;  
  }  
};  
  
let rx_fut = async {  
  while let Some(value) = rx.recv().await {  
    println!("received '{value}'");  
  }  
};  
  
trpl::join(tx_fut, rx_fut).await;
```

- can be cloned
- join3 exists too

Joining More Async Blocks



- ```
use std::pin::Pin;
// -- snip --
let futures: Vec<Pin<Box<dyn Future<Output = ()>>>> =
 vec![Box::pin(tx1_fut), Box::pin(rx_fut), Box::pin(tx_fut)];
```
- ```
use std::pin::{Pin, pin};
// -- snip --
    let tx1_fut = pin!(async move {
        // --snip--
    });

    let rx_fut = pin!(async {
        // --snip--
    });

    let tx_fut = pin!(async move {
        // --snip--
    });

    let futures: Vec<Pin<&mut dyn Future<Output = ()>>> =
        vec![tx1_fut, rx_fut, tx_fut];
```

Using Macro for Join



- ```
let a = async { 1u32 };
let b = async { "Hello!" };
let c = async { true };

let (a_result, b_result, c_result) = trpl::join!(a, b, c);
println!("{a_result}, {b_result}, {c_result}");
```
- 
- Watch: different types!

# Races



- ```
let slow = async {
  println!("'slow' started.");
  trpl::sleep(Duration::from_millis(100)).await;
  println!("'slow' finished.");
};

let fast = async {
  println!("'fast' started.");
  trpl::sleep(Duration::from_millis(50)).await;
  println!("'fast' finished.");
};

trpl::race(slow, fast).await;
```

Yield – Making Slow Process



- ```
fn slow(name: &str, ms: u64) {
 thread::sleep(Duration::from_millis(ms));
 println!("'{name}' ran for {ms}ms");
}
```

-

# Yield – Running Slow Process



```
let a = async {
 println!("'a' started.");
 slow("a", 30);
 slow("a", 10);
 slow("a", 20);
 trpl::sleep(Duration::from_millis(50)).await;
 println!("'a' finished.");
};

let b = async {
 println!("'b' started.");
 slow("b", 75);
 slow("b", 10);
 slow("b", 15);
 slow("b", 350);
 trpl::sleep(Duration::from_millis(50)).await;
 println!("'b' finished.");
};

trpl::race(a, b).await;
```

# Yield – Output of Slow Process 😞



- 'a' started.  
'a' ran for 30ms  
'a' ran for 10ms  
'a' ran for 20ms  
'b' started.  
'b' ran for 75ms  
'b' ran for 10ms  
'b' ran for 15ms  
'b' ran for 350ms  
'a' finished.



# Yield – Give Up Immediately



```
let a = async {
 println!("'a' started.");
 slow("a", 30);
 trpl::yield_now().await;
 slow("a", 10);
 trpl::yield_now().await;
 slow("a", 20);
 trpl::yield_now().await;
 println!("'a' finished.");
};
```

```
let b = async {
 println!("'b' started.");
 slow("b", 75);
 trpl::yield_now().await;
 slow("b", 10);
 trpl::yield_now().await;
 slow("b", 15);
 trpl::yield_now().await;
 slow("b", 350);
 trpl::yield_now().await;
 println!("'b' finished.");
};
```

// yield is better then sleep for 1ns

# Streams 1



- **use trpl::StreamExt;**

```
fn main() {
 trpl::run(async {
 let values = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
 let iter = values.iter().map(|n| n * 2);
 let mut stream = trpl::stream_from_iter(iter);

 while let Some(value) = stream.next().await {
 println!("The value was: {value}");
 }
 });
}
```

# Streams 2 – More Haskellish ☺



- **use `trpl::StreamExt`;**

```
fn main() {
 trpl::run(async {
 let values = 1..101;
 let iter = values.map(|n| n * 2);
 let stream = trpl::stream_from_iter(iter);

 let mut filtered =
 stream.filter(|value| value % 3 == 0 || value % 5 == 0);

 while let Some(value) = filtered.next().await {
 println!("The value was: {value}");
 }
 });
}
```

# OOP in Rust



- ```
pub struct AveragedCollection {    // struct can be used  
    list: Vec<i32>,           // field is private  
    average: f64,             // also private  
}
```
- We can use the structure, but manipulation of its fields can be only via methods...

Methods for Average 1



- ```
impl AveragedCollection {
 pub fn add(&mut self, value: i32) {
 self.list.push(value);
 self.update_average();
 }

 pub fn remove(&mut self) -> Option<i32> {
 let result = self.list.pop();
 match result {
 Some(value) => {
 self.update_average();
 Some(value)
 }
 None => None,
 }
 }
}
```

# Methods for Average 2



- ```
pub fn average(&self) -> f64 {  
    self.average  
}  
  
// also private method  
fn update_average(&mut self) {  
    let total: i32 = self.list.iter().sum();  
    self.average = total as f64 / self.list.len() as f64;  
}  
}
```

Inheritance



- No
- But we have traits for some replacement
 - Type classes in Haskell
 - Interfaces with default behavior in some OOPL
- Note: seems inheritance is not used as much as expected anyway...

Traits Example



- In some lib.rs
- ```
pub trait Draw {
 fn draw(&self);
}
```
- ```
pub struct Screen {  
    pub components: Vec<Box<dyn Draw>>,  
}
```
- ```
impl Screen {
 pub fn run(&self) {
 for component in self.components.iter() {
 component.draw();
 }
 }
}
```



# Watch!!



- This is something different – homogeneous type
- ```
pub struct Screen<T: Draw> {  
    pub components: Vec<T>,  
}  
  
impl<T> Screen<T>  
where  
    T: Draw,  
{  
    pub fn run(&self) {  
        for component in self.components.iter() {  
            component.draw();  
        }  
    }  
}
```

Pattern Matching



- Pattern must be a composition of:
 - Literals
 - Destructured arrays, enums, structs, or tuples
 - Variables
 - Wildcards
 - Placeholders

match



- `match VALUE {
 PATTERN => EXPRESSION,
 PATTERN => EXPRESSION,
 PATTERN => EXPRESSION,
}`
- **`match x {
 None => None,
 Some(i) => Some(i + 1),
}`**

Matching Literals



- ```
let x = 1;
match x {
 1 => println!("one"),
 2 => println!("two"),
 3 => println!("three"),
 _ => println!("anything"),
}
```

# Matching Named Variables



- ```
let x = Some(5);  
let y = 10;  
  
match x {  
    Some(50) => println!("Got 50"),  
    Some(y) => println!("Matched, y = {y}"),  
    _ => println!("Default case, x = {x:?}"),  
}  
  
println!("at the end: x = {x:?}, y = {y}");
```

Multiple Patterns



- ```
let x = 1;

match x {
 1 | 2 => println!("one or two"),
 3 => println!("three"),
 _ => println!("anything"),
}
```

# Matching Ranges



- ```
let x = 5;
match x {
  1..=5 => println!("one through five"),
  _ => println!("something else"),
}
```
- ```
let x = 'c';
match x {
 'a'..'j' => println!("early ASCII letter"),
 'k'..'z' => println!("late ASCII letter"),
 _ => println!("something else"),
}
```

# Match with Condition 1



- `let num = Some(4);`

```
match num {
 Some(x) if x % 2 == 0 => println!("The number {x} is even"),
 Some(x) => println!("The number {x} is odd"),
 None => (),
}
```



## Match with Condition 2



- ```
fn main() {  
    let x = Some(5);  
    let y = 10;  
  
    match x {  
        Some(50) => println!("Got 50"),  
        Some(n) if n == y => println!("Matched, n = {n}"),  
        _ => println!("Default case, x = {x:?}"),  
    }  
  
    println!("at the end: x = {x:?}, y = {y}");  
}
```

Match with Condition 3



- ```
let x = 4;
let y = false;

match x {
 4 | 5 | 6 if y => println!("yes"),
 _ => println!("no"),
}
```

# if let



```
• fn main() {
 let favorite_color: Option<&str> = None;
 let is_tuesday = false;
 let age: Result<u8, _> = "34".parse();

 if let Some(color) = favorite_color {
 println!("Using your favorite color, {color}, as the background");
 } else if is_tuesday {
 println!("Tuesday is green day!");
 } else if let Ok(age) = age {
 if age > 30 {
 println!("Using purple as the background color");
 } else {
 println!("Using orange as the background color");
 }
 } else {
 println!("Using blue as the background color");
 }
}
```

# while let



- ```
let (tx, rx) = std::sync::mpsc::channel();
std::thread::spawn(move || {
    for val in [1, 2, 3] {
        tx.send(val).unwrap();
    }
});

while let Ok(value) = rx.recv() {
    println!("{value}");
}
```

for



- ```
let v = vec!['a', 'b', 'c'];
for (index, value) in v.iter().enumerate() {
 println!("{value} is at index {index}");
}
```
- ```
$ cargo run  
Compiling patterns v0.1.0 (file:///projects/patterns)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.52s  
Running `target/debug/patterns`  
a is at index 0  
b is at index 1  
c is at index 2
```

let



- `let PATTERN = EXPRESSION;`
- `let (x, y, z) = (1, 2, 3);`

Function Parameters



- **fn print_coordinates(&(x, y): &(i32, i32)) {
 println!("Current location: ({x}, {y})");
}**

```
fn main() {  
    let point = (3, 5);  
    print_coordinates(&point);  
}
```

Destructuring struct 1



- ```
struct Point {
 x: i32,
 y: i32,
}

fn main() {
 let p = Point { x: 0, y: 7 };

 let Point { x: a, y: b } = p;
 assert_eq!(0, a);
 assert_eq!(7, b);
}
```



# Destructuring struct 2



- ```
struct Point {  
    x: i32,  
    y: i32,  
}
```



```
fn main() {  
    let p = Point { x: 0, y: 7 };  
  
    let Point { x, y } = p;  
    assert_eq!(0, x);  
    assert_eq!(7, y);  
}
```

Destructuring struct 3



- ```
fn main() {
 let p = Point { x: 0, y: 7 };

 match p {
 Point { x, y: 0 } => println!("On the x axis at {x}"),
 Point { x: 0, y } => println!("On the y axis at {y}"),
 Point { x, y } => {
 println!("On neither axis: ({x}, {y})");
 }
 }
}
```

# Destructuring enum



```
• enum Message {
 Quit,
 Move { x: i32, y: i32 },
 Write(String),
 ChangeColor(i32, i32, i32),
}
fn main() {
 let msg = Message::ChangeColor(0, 160, 255);
 match msg {
 Message::Quit => {
 println!("The Quit variant has no data to destructure.");
 }
 Message::Move { x, y } => {
 println!("Move in the x direction {x} and in the y direction {y}");
 }
 Message::Write(text) => {
 println!("Text message: {text}");
 }
 Message::ChangeColor(r, g, b) => {
 println!("Change color to red {r}, green {g}, and blue {b}");
 }
 }
}
```

# Destructuring Nested struct and enum



```
• enum Color {
 Rgb(i32, i32, i32),
 Hsv(i32, i32, i32),
}
enum Message {
 Quit,
 Move { x: i32, y: i32 },
 Write(String),
 ChangeColor(Color),
}
fn main() {
 let msg = Message::ChangeColor(Color::Hsv(0, 160, 255));
 match msg {
 Message::ChangeColor(Color::Rgb(r, g, b)) => {
 println!("Change color to red {r}, green {g}, and blue {b}");
 }
 Message::ChangeColor(Color::Hsv(h, s, v)) => {
 println!("Change color to hue {h}, saturation {s}, value {v}");
 }
 _ => (),
 }
}
```

# Ignoring Values 1



- ```
fn foo(_: i32, y: i32) {  
    println!("This code only uses the y parameter: {y}");  
}  
  
fn main() {  
    foo(3, 4);  
}
```

Ignoring Values 2



- ```
let mut setting_value = Some(5);
let new_setting_value = Some(10);

match (setting_value, new_setting_value) {
 (Some(_), Some(_)) => {
 println!("Can't overwrite an existing customized value");
 }
 _ => {
 setting_value = new_setting_value;
 }
}

println!("setting is {setting_value:?}");
```

# Ignoring Values 3



- ```
let numbers = (2, 4, 8, 16, 32);  
  
match numbers {  
    (first, _, third, _, fifth) => {  
        println!("Some numbers: {first}, {third}, {fifth}");  
    }  
}
```

Unused Variable



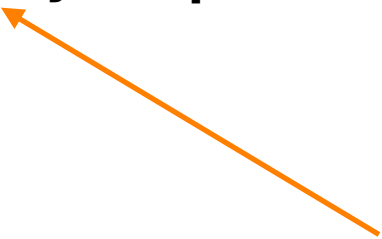
- ```
fn main() {
 let _x = 5;
 let y = 10;
}
```
- If a variable is not used compiler provides a warning
- Unused variables remove such a warning if it is expected to be used later or for debugging



# Remaining Parts 1



- ```
struct Point {  
    x: i32,  
    y: i32,  
    z: i32,  
}  
  
let origin = Point { x: 0, y: 0, z: 0 };  
  
match origin {  
    Point { x, .. } => println!("x is {x}"),  
}
```



Remaining Parts 2



- ```
fn main() {
 let numbers = (2, 4, 8, 16, 32);

 match numbers {
 (first, .., last) => {
 println!("Some numbers: {first}, {last}");
 }
 }
}
```

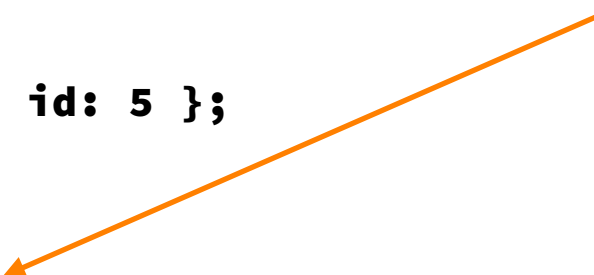
# @ Bindigs



- enum Message {  
    Hello { id: i32 },  
}

```
let msg = Message::Hello { id: 5 };

match msg {
 Message::Hello {
 id: id_variable @ 3..=7,
 } => println!("Found an id in range: {id_variable}"),
 Message::Hello { id: 10..=12 } => {
 println!("Found an id in another range")
 }
 Message::Hello { id } => println!("Found some other id: {id}"),
}
```



# Advanced Features



- See documentation
  - Unsafe code
  - Traits
  - Types
  - Functions and Closures
  - Macros

# Type Synonyms



- Haskell like
- E. g.
  - **type Kilometers = i32;**

# Never Returning Function Type



- **fn bar() -> ! {  
    // --snip--  
}**

# Functions as Parameters



- ```
fn add_one(x: i32) -> i32 {  
    x + 1  
}  
  
fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {  
    f(arg) + f(arg)  
}  
  
fn main() {  
    let answer = do_twice(add_one, 5);  
  
    println!("The answer is: {answer}");  
}
```

Returning Closures



- ```
fn returns_closure() -> Box<dyn Fn(i32) -> i32> {
 Box::new(|x| x + 1)
}

fn returns_initialized_closure(init: i32) -> Box<dyn Fn(i32) -> i32> {
 Box::new(move |x| x + init)
}
```





Thanks!